



一、为什么要用多线程

多线程 (Multithreading) 允许程序 **同时执行多个任务**，提升资源利用率与系统吞吐量。

应用场景：

- Web 服务器处理并发请求 (如 Tomcat、Netty)
- 异步任务 (日志、短信、通知)
- 并行计算 (大数据、AI、图像处理)
- 分布式系统 (任务调度、事件驱动)



二、Java 线程模型概述

Java 提供了对操作系统线程的 **面向对象封装**。

Java Thread → JVM Thread → OS Thread

一个 Java 线程在 JVM 层面映射为一个 **本地操作系统线程 (Native Thread)**，由操作系统负责调度执行。



三、创建线程的三种方式

① 继承 Thread 类

```
public class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread running...");  
    }  
    public static void main(String[] args) {  
        new MyThread().start();  
    }  
}
```

② 实现 Runnable 接口

```
public class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable running...");
    }
    public static void main(String[] args) {
        new Thread(new MyRunnable()).start();
    }
}
```

③ 实现 Callable + Future (有返回值)

```
import java.util.concurrent.*;

public class MyCallable implements Callable<Integer> {
    public Integer call() {
        return 123;
    }
    public static void main(String[] args) throws Exception {
        ExecutorService executor =
        Executors.newSingleThreadExecutor();
        Future<Integer> result = executor.submit(new
        MyCallable());
        System.out.println(result.get());
        executor.shutdown();
    }
}
```



四、线程的生命周期

线程的状态 (Thread.State) :

NEW → RUNNABLE → BLOCKED / WAITING / TIMED_WAITING → TERMINATED

状态	含义
NEW	尚未启动
RUNNABLE	就绪/运行中
BLOCKED	被锁阻塞

状态	含义
WAITING	等待其他线程通知
TIMED_WAITING	有超时等待
TERMINATED	线程执行完毕

常见方法：

```
thread.start();        // 启动线程
thread.join();         // 等待线程执行结束
thread.sleep(1000);    // 线程睡眠
thread.yield();        // 让出CPU
thread.interrupt();    // 中断线程
```

五、线程同步机制 (Synchronized)

① 同步方法

```
public synchronized void add() {
    count++;
}
```

② 同步代码块

```
synchronized (this) {
    count++;
}
```

③ 静态同步

```
public static synchronized void add() { ... }
```

锁的粒度：

- 对象锁（实例方法或代码块）
- 类锁（静态方法）

六、volatile 关键字

`volatile` 保证：

- **可见性**：一个线程修改的变量，立即对其他线程可见；
- **有序性**：禁止指令重排。

不保证原子性！

示例：

```
volatile boolean running = true;
```

七、Lock 接口（显式锁）

`java.util.concurrent.locks` 包提供更灵活的锁控制。

```
import java.util.concurrent.locks.*;

Lock lock = new ReentrantLock();

lock.lock();
try {
    // 临界区
} finally {
    lock.unlock();
}
```

✓ 可重入锁 ReentrantLock

- 可中断；
- 可限时；
- 支持公平/非公平锁。

```
ReentrantLock lock = new ReentrantLock(true); // 公平锁
```

✓ 读写锁 ReadWriteLock

```
ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();
rwLock.readLock().lock();
rwLock.writeLock().lock();
```

✖ 八、线程间通信

① wait / notify

```
synchronized (obj) {  
    while (!condition) {  
        obj.wait();  
    }  
    obj.notify();  
}
```

② Condition (基于 Lock)

```
Condition condition = lock.newCondition();  
  
lock.lock();  
try {  
    condition.await();    // 等待  
    condition.signal();   // 唤醒  
} finally {  
    lock.unlock();  
}
```

③ 阻塞队列 (BlockingQueue)

生产者 - 消费者模式:

```
BlockingQueue<String> queue = new ArrayBlockingQueue<>(10);  
  
queue.put("task");  
String task = queue.take();
```

⚡ 九、线程池 (Executor 框架)

使用线程池代替手动创建线程。

```
ExecutorService pool = Executors.newFixedThreadPool(10);  
pool.execute(() -> System.out.println("任务执行"));  
pool.shutdown();
```

推荐使用:

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(
    5, 10, 60, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(100),
    Executors.defaultThreadFactory(),
    new ThreadPoolExecutor.AbortPolicy()
);
```

线程池优点：

- 降低线程创建开销；
- 提高系统响应速度；
- 可统一管理并发策略。



十、常用并发工具类（JUC 包）

工具类	功能
CountDownLatch	倒计时同步器（等待多个线程完成）
CyclicBarrier	多线程循环栅栏
Semaphore	控制并发线程数
ConcurrentHashMap	高效并发Map
CopyOnWriteArrayList	线程安全List
AtomicInteger	原子变量，CAS实现



十一、CAS与原子操作

CAS (Compare-And-Swap)：

无锁并发机制，基于 CPU 指令实现。

```
AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet(); // 原子自增
```

优点：

- 无锁；
- 高并发下性能优。

缺点：

- 自旋开销;
- ABA问题 (用 `AtomicStampedReference` 解决) 。

十二、高级话题

✓ ThreadLocal (线程局部变量)

```
ThreadLocal<Integer> local = ThreadLocal.withInitial(() -> 0);  
local.set(100);
```

每个线程独立副本，常用于数据库连接、会话信息存储。

✓ Fork/Join框架 (并行计算)

```
ForkJoinPool pool = new ForkJoinPool();  
pool.submit(new RecursiveTask<Long>() { ... });
```

适用于分治算法 (如归并排序、矩阵乘法) 。

十三、常见面试与实战问题

问题	要点
synchronized与Lock区别	粒度、可重入性、公平性
volatile保证什么	可见性、有序性
线程池拒绝策略	Abort、CallerRuns、Discard、DiscardOldest
死锁产生条件	互斥、占有且等待、不可剥夺、循环等待
如何避免死锁	锁顺序、定时锁、减少锁粒度

十四、推荐学习顺序

- 1 理解线程模型与生命周期
- 2 掌握 `synchronized` / `volatile` / `Lock`
- 3 熟悉 JUC 工具类与线程池
- 4 学习并发容器与原子类
- 5 实战：生产者消费者、线程池监控、死锁检测
- 6 掌握 JVM 内存模型 (JMM) 与 Happens-Before



十五、可视化图示（线程执行模型）

（可生成一张图，展示线程从创建 → 就绪 → 运行 → 等待 → 终止的全过程）

是否希望我为你生成一张「Java 多线程全景图」？

内容包括线程生命周期、同步机制（锁、volatile）、JUC 工具类与线程池之间的关系。

111

第一部分：多线程基础概念

1. 为什么需要多线程？

- **提高资源利用率**：在等待 I/O（如网络、磁盘）时，CPU 可以执行其他任务。
- **提高系统吞吐量**：在多核 CPU 上，多个线程可以真正并行执行，加快处理速度。
- **改善用户体验**：在图形界面程序中，使用后台线程执行耗时任务，可以避免界面“卡死”。

2. 进程 vs. 线程

- **进程**：操作系统**资源分配**的基本单位。一个进程包含一个或多个线程。进程之间相互独立。
- **线程**：CPU **调度和执行**的基本单位。线程是进程内的一个执行流，共享进程的堆和方法区，但拥有自己独立的程序计数器、虚拟机栈和本地方法栈。

简单比喻：

- **进程** = 一个工厂车间。
- **线程** = 车间里的工人。
- **内存空间** = 车间里的公共空间（如原料库，工人们共享）。
- **栈内存** = 每个工人自己的工作台（独立，互不干扰）。

第二部分：创建和管理线程

1. 继承 `Thread` 类

java

```
public class MyThread extends Thread {
    @Override
    public void run() {
        // 线程要执行的任务
        System.out.println("线程运行了: " +
Thread.currentThread().getName());
    }

    public static void main(String[] args) {
        MyThread thread = new MyThread();
        thread.start(); // 启动线程, 注意是 start() 而不是 run()
    }
}
```

2. 实现 `Runnable` 接口 (推荐)

java

```
public class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("线程运行了: " +
Thread.currentThread().getName());
    }

    public static void main(String[] args) {
        Thread thread = new Thread(new MyRunnable());
        thread.start();

        // 使用 Lambda 表达式 (更简洁)
        Thread lambdaThread = new Thread(() -> {
            System.out.println("Lambda线程: " +
Thread.currentThread().getName());
        });
        lambdaThread.start();
    }
}
```

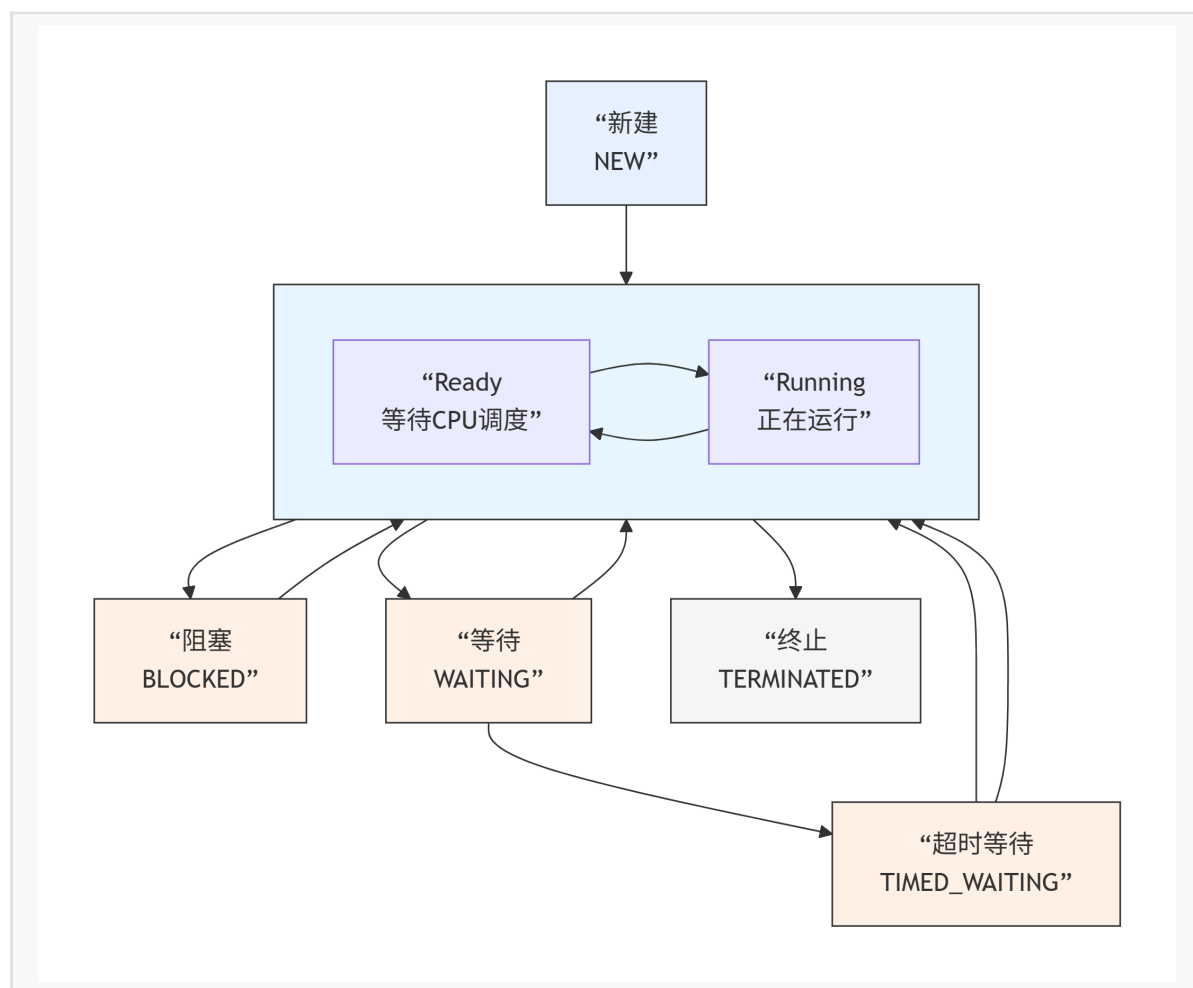
为什么推荐 `Runnable`？

- 避免单继承的局限性（Java 是单继承）。
- 更适合多个线程共享同一个任务逻辑。
- 线程池可以更方便地管理 `Runnable` 任务。

3. 线程的生命周期（状态）

线程在其生命周期中会处于不同的状态：

下图清晰地展示了Java线程的完整生命周期与状态转换：



- **NEW (新建)**：线程被创建，但尚未调用 `start()`。
- **RUNNABLE (可运行)**：调用了 `start()`，正在等待 CPU 时间片或正在运行。
- **BLOCKED (阻塞)**：线程等待获取一个**同步锁**，以进入 `synchronized` 方法或代码块。
- **WAITING (等待)**：线程进入等待状态，需要被其他线程**显式唤醒**。如 `object.wait()`, `thread.join()`。

- **TIMED_WAITING (超时等待)**：在指定的时间后会自动唤醒。如 `Thread.sleep(millis)`, `object.wait(timeout)`。
 - **TERMINATED (终止)**：线程执行完毕或因异常退出。
-

第三部分：线程同步与锁

当多个线程访问**共享资源**时，可能会发生**数据竞争**，导致数据不一致。同步是保证线程安全的手段。

1. `synchronized` 关键字

Java 中最基本的互斥同步手段。

a) 同步实例方法

java

```
public synchronized void add(int value) {  
    this.count += value;  
}  
// 锁是当前实例对象 (this)
```

b) 同步静态方法

java

```
public static synchronized void staticMethod() {  
    // ...  
}  
// 锁是当前类的Class对象 (MyClass.class)
```

c) 同步代码块

java

```
public void method() {  
    // 非同步代码...  
  
    synchronized (lockObject) { // 锁是 lockObject  
        // 对共享资源的操作  
    }  
  
    // 非同步代码...  
}
```

2. `volatile` 关键字

- 保证变量的**可见性**：一个线程修改了 `volatile` 变量，新值对其他线程**立即可见**。
- **禁止指令重排序**。
- **不保证原子性**（如 `count++` 不是原子操作）。

适用场景：作为状态标志位。

java

```
public class ShutdownThread extends Thread {  
    private volatile boolean shutdownRequested = false;  
  
    public void shutdown() {  
        shutdownRequested = true;  
    }  
  
    @Override  
    public void run() {  
        while (!shutdownRequested) {  
            // 执行任务...  
        }  
    }  
}
```

第四部分：Java 内存模型与 `java.util.concurrent` 包

`java.util.concurrent` (JUC) 包提供了强大、高效的并发工具类，是现代 Java 并发编程的首选。

1. 原子类

解决 `count++` 这类复合操作的原子性问题。

java

```
import java.util.concurrent.atomic.AtomicInteger;

public class Counter {
    private AtomicInteger count = new AtomicInteger(0);

    public void increment() {
        count.incrementAndGet(); // 原子性的 ++count
    }

    public int getCount() {
        return count.get();
    }
}
```

原理：基于 CAS 操作。

2. 显式锁 `ReentrantLock`

比 `synchronized` 更灵活、功能更丰富的锁。

java

```
import java.util.concurrent.locks.ReentrantLock;

public class ExplicitLockExample {
    private final ReentrantLock lock = new ReentrantLock();
    private int count = 0;

    public void increment() {
        lock.lock(); // 手动加锁
        try {
```

```
        count++;
    } finally {
        lock.unlock(); // 必须在finally块中解锁，确保锁释放
    }
}
```

优势：

- **可中断的锁获取：** `lockInterruptibly()`
- **超时获取锁：** `tryLock(long time, TimeUnit unit)`
- **公平锁：** `new ReentrantLock(true)`

3. 并发集合

代替传统的同步集合（如 `vector`, `Hashtable`），性能更高。

- `ConcurrentHashMap`：高效的并发 `HashMap`。
- `CopyOnWriteArrayList`：读多写少的并发 `List`。
- `ConcurrentLinkedQueue`：高效的并发队列。

4. 线程池

为什么需要线程池？

- 降低资源消耗：减少创建和销毁线程的开销。
- 提高响应速度：任务到达时，无需等待线程创建。
- 提高线程的可管理性：可以统一分配、调优和监控。

使用 `Executors` 工厂类创建线程池：

java

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadPoolExample {
    public static void main(String[] args) {
        // 创建固定大小的线程池
        ExecutorService executor =
            Executors.newFixedThreadPool(5);

        // 提交任务
```

```

        for (int i = 0; i < 10; i++) {
            final int taskId = i;
            executor.execute(() -> {
                System.out.println("执行任务: " + taskId + ", 线程: " + Thread.currentThread().getName());
            });
        }

        // 关闭线程池（不再接受新任务，等待已提交任务完成）
        executor.shutdown();
    }
}

```

重要线程池类型：

- `newFixedThreadPool(n)`：固定大小线程池。
- `newCachedThreadPool()`：可缓存线程池，线程数自动扩张/收缩。
- `newSingleThreadExecutor()`：单线程线程池，保证任务顺序执行。
- `newScheduledThreadPool(n)`：支持定时及周期性任务执行的线程池。

第五部分：高级同步工具

1. `CountDownLatch`（倒计时门闩）

允许一个或多个线程等待其他线程完成操作。

java

```

public class CountDownLatchExample {
    public static void main(String[] args) throws InterruptedException {
        CountDownLatch latch = new CountDownLatch(3); // 初始计数为3

        for (int i = 0; i < 3; i++) {
            new Thread(() -> {
                System.out.println("子线程" + Thread.currentThread().getName() + "执行");
                latch.countDown(); // 计数减1
            }).start();
        }

        latch.await(); // 主线程等待，直到计数变为0
    }
}

```

```
        System.out.println("所有子线程执行完毕，主线程继续");
    }
}
```

2. `CyclicBarrier` (循环栅栏)

让一组线程到达一个屏障时被阻塞，直到最后一个线程到达屏障时，屏障才会开门，所有被屏障拦截的线程才会继续运行。

java

```
public class CyclicBarrierExample {
    public static void main(String[] args) {
        CyclicBarrier barrier = new CyclicBarrier(3, () -> {
            System.out.println("所有线程都到达屏障，执行屏障动作");
        });

        for (int i = 0; i < 3; i++) {
            new Thread(() -> {
                try {
                    System.out.println("线程" +
                        Thread.currentThread().getName() + "到达屏障");
                    barrier.await(); // 等待其他线程
                    System.out.println("线程" +
                        Thread.currentThread().getName() + "继续执行");
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }).start();
        }
    }
}
```

3. `Semaphore` (信号量)

控制同时访问特定资源的线程数量，用于流量控制。

java

```
public class SemaphoreExample {
    public static void main(String[] args) {
```



```
Semaphore semaphore = new Semaphore(3); // 允许3个线程同时访问

for (int i = 0; i < 10; i++) {
    new Thread(() -> {
        try {
            semaphore.acquire(); // 获取许可
            System.out.println("线程" +
Thread.currentThread().getName() + "获得许可，正在执行");
            Thread.sleep(2000); // 模拟业务操作
            System.out.println("线程" +
Thread.currentThread().getName() + "释放许可");
            semaphore.release(); // 释放许可
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }).start();
}
}
```

第六部分：最佳实践与常见陷阱

1. 避免死锁

死锁产生的四个必要条件（缺一不可）：

- **互斥条件**：资源一次只能被一个线程占用。
- **请求与保持条件**：一个线程因请求资源而阻塞时，对已获得的资源保持不放。
- **不剥夺条件**：线程已获得的资源在未使用完之前不能被强行剥夺。
- **循环等待条件**：若干线程之间形成一种头尾相接的循环等待资源关系。

预防死锁：破坏上述条件之一，例如**按固定的顺序获取锁**。

2. 性能考量

- **减小锁的粒度**：只锁必要的代码块。
- **减少锁的持有时间**：尽快释放锁。
- **读多写少时使用读写锁**：`ReentrantReadWriteLock`。
- **考虑使用无锁编程**：原子变量。

3. 其他陷阱

- **上下文切换开销**：线程不是越多越好。
- **资源泄漏**：确保在 `finally` 块中释放锁、关闭连接等资源。
- **活锁**：线程不断重复执行相同的操作，但总是失败。

总结

Java 多线程的学习路径：

1. **理解基础**：线程创建、生命周期、基本同步（`synchronized`, `volatile`）。
2. **掌握 JUC**：原子变量、显式锁、并发集合、线程池。
3. **熟练使用高级工具**：`CountDownLatch`, `CyclicBarrier`, `Semaphore`。
4. **深入原理**：JMM（Java内存模型）、Happens-Before、CAS、AQS。
5. **实践与调优**：诊断死锁、性能优化、避免常见陷阱。

111

Java 多线程教程（从入门到实战）

Java 多线程是并发编程的核心，通过充分利用 CPU 多核资源，可提升程序吞吐量（如高并发接口）和响应速度（如后台任务异步执行）。本教程从基础概念到实战应用，循序渐进讲解 Java 多线程的核心知识，覆盖线程创建、同步机制、并发工具、性能优化等关键内容。

一、多线程核心概念（先理解本质）

在学习代码前，需先理清以下核心概念，避免后续混淆：

概念	定义与说明
线程（Thread）	CPU 调度的最小单位，共享进程的内存空间（堆、方法区），拥有独立的栈和程序计数器
进程（Process）	操作系统资源分配的最小单位，一个进程可包含多个线程（如 Java 程序默认 1 个主线程）
并发（Concurrency）	多个线程“交替执行”（单核 CPU 模拟多任务，如秒杀系统多用户同时下单）
并行（Parallelism）	多个线程“同时执行”（多核 CPU 真正并行，如大数据处理多线程拆分计算）

概念	定义与说明
线程安全	多线程并发访问共享资源时，程序执行结果与单线程一致（无脏读、重复计算等问题）
共享资源	多线程共同访问的变量 / 对象（如静态变量、堆内存中的对象字段）
竞态条件（Race Condition）	多线程同时修改共享资源，导致执行结果依赖线程执行顺序（线程不安全的根源）
上下文切换	CPU 从一个线程切换到另一个线程的开销（保存线程状态、恢复线程状态）

二、线程的创建与启动（4 种方式）

Java 提供 4 种创建线程的方式，核心是“定义任务（要执行的逻辑）”和“启动线程”，推荐优先使用线程池（方式 4）。

方式 1：继承 Thread 类（基础方式）

通过继承 `java.lang.Thread` 类，重写 `run()` 方法定义任务，调用 `start()` 方法启动线程（**不能直接调用 `run()` 方法**，否则相当于普通方法调用，不会启动新线程）。

示例代码：

java

运行

```
// 1. 继承 Thread 类
class MyThread extends Thread {
    // 2. 重写 run() 方法（线程要执行的任务）
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
```

```

        System.out.println(Thread.currentThread().getName() +
": " + i);
        try {
            Thread.sleep(100); // 线程休眠 100ms, 释放 CPU 资源
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

// 测试
public class ThreadDemo {
    public static void main(String[] args) {
        // 3. 创建线程实例
        MyThread thread1 = new MyThread();
        MyThread thread2 = new MyThread();

        // 4. 设置线程名称 (可选)
        thread1.setName("线程A");
        thread2.setName("线程B");

        // 5. 启动线程 (调用 start(), JVM 会自动调用 run())
        thread1.start();
        thread2.start();

        // 主线程执行逻辑
        for (int i = 0; i < 5; i++) {
            System.out.println("主线程: " + i);
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```

输出结果（线程交替执行，顺序不固定）：

plaintext

```
主线程：0  
线程A：0  
线程B：0  
主线程：1  
线程A：1  
线程B：1  
...
```

优缺点：

- 优点：实现简单，直接继承 `Thread` 即可；
- 缺点：Java 单继承限制，无法再继承其他类；任务与线程耦合，不便于复用。

方式 2：实现 `Runnable` 接口（推荐基础方式）

`Runnable` 是函数式接口（仅含 `run()` 方法），通过实现该接口定义任务，再将任务传入 `Thread` 类启动线程（解耦任务与线程）。

示例代码：

java

运行

```
// 1. 实现 Runnable 接口（定义任务）  
class MyRunnable implements Runnable {  
    @Override  
    public void run() {  
        for (int i = 0; i < 5; i++) {  
            System.out.println(Thread.currentThread().getName() +  
": " + i);  
        }  
    }  
}
```

```

        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

// 测试
public class RunnableDemo {
    public static void main(String[] args) {
        // 2. 创建任务实例
        Runnable task = new MyRunnable();

        // 3. 将任务传入 Thread, 启动线程
        Thread thread1 = new Thread(task, "线程A");
        Thread thread2 = new Thread(task, "线程B");

        thread1.start();
        thread2.start();
    }
}

```

Lambda 简化 (JDK 8+, 推荐) :

`Runnable` 是函数式接口, 可直接用 Lambda 表达式省略接口实现类:

java

运行

```

public class LambdaRunnableDemo {
    public static void main(String[] args) {
        // Lambda 表达式定义任务
        Runnable task = () -> {
            for (int i = 0; i < 5; i++) {

```

```

System.out.println(Thread.currentThread().getName() + ": " + i);
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
};

new Thread(task, "线程A").start();
new Thread(task, "线程B").start();
}
}

```

优缺点:

- 优点: 无单继承限制, 任务与线程解耦, 可复用;
- 缺点: `run()` 方法无返回值, 无法抛出受检异常 (需在方法内捕获)。

方式 3: 实现 Callable 接口 (有返回值 / 可抛异常)

`Callable` 接口 (JDK 5+) 支持任务返回结果和抛出受检异常, 需配合 `FutureTask` 类 (实现 `RunnableFuture` 接口) 使用, 适用于需要获取任务执行结果的场景 (如异步计算)。

核心流程:

1. 实现 `Callable<V>` 接口 (`V` 为返回值类型), 重写 `call()` 方法;
2. 创建 `Callable` 实例, 包装为 `FutureTask` (兼具 `Runnable` 和 `Future` 特性);
3. 将 `FutureTask` 传入 `Thread` 启动线程;
4. 调用 `FutureTask.get()` 方法获取返回值 (会阻塞, 直到任务执行完成)。

示例代码:

java

运行

```
import java.util.concurrent.Callable;
import java.util.concurrent.FutureTask;

// 1. 实现 callable 接口 (V 为返回值类型, 此处为 Integer)
class MyCallable implements Callable<Integer> {
    private int num;

    public MyCallable(int num) {
        this.num = num;
    }

    // 2. 重写 call() 方法 (有返回值, 可抛异常)
    @Override
    public Integer call() throws Exception {
        int sum = 0;
        for (int i = 1; i <= num; i++) {
            sum += i;
            Thread.sleep(50);
        }
        System.out.println(Thread.currentThread().getName() + " 计算完成, sum=" + sum);
        return sum;
    }
}

// 测试
public class CallableDemo {
    public static void main(String[] args) throws Exception {
        // 3. 创建 callable 任务实例
        Callable<Integer> task1 = new MyCallable(10);
        Callable<Integer> task2 = new MyCallable(20);

        // 4. 包装为 FutureTask
        FutureTask<Integer> futureTask1 = new FutureTask<>(task1);
        FutureTask<Integer> futureTask2 = new FutureTask<>(task2);

        // 5. 启动线程
        new Thread(futureTask1, "计算线程1").start();
        new Thread(futureTask2, "计算线程2").start();

        // 6. 获取返回值 (get() 会阻塞, 直到任务完成)
```



```
Integer result1 = futureTask1.get();
Integer result2 = futureTask2.get();

System.out.println("线程1返回结果: " + result1); // 输出 55
System.out.println("线程2返回结果: " + result2); // 输出 210
    }
}
```

优缺点:

- 优点: 有返回值, 可抛出受检异常, 适用于异步计算场景;
- 缺点: 实现稍复杂, `get()` 方法会阻塞, 需注意线程阻塞风险。

方式 4: 线程池 (ExecutorService, 生产环境首选)

直接创建线程 (方式 1-3) 存在明显缺陷: 线程创建 / 销毁开销大、无限制创建线程可能导致 OOM、无法统一管理线程。**线程池**通过复用线程、统一管理线程生命周期, 是生产环境中最推荐的多线程使用方式 (JDK 5+ 提供 `java.util.concurrent` 并发工具包)。

核心接口与类:

- `ExecutorService`: 线程池核心接口, 定义线程池操作 (提交任务、关闭线程池等);
- `Executors`: 线程池工具类, 提供静态方法快速创建线程池 (如 `newFixedThreadPool`、`newCachedThreadPool`);
- `ThreadPoolExecutor`: 线程池核心实现类, 支持自定义线程池参数 (推荐生产环境直接使用, 而非 `Executors`)。

示例 1: 使用 Executors 快速创建线程池 (入门)

java

运行

```

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadPoolDemo {
    public static void main(String[] args) {
        // 1. 创建固定大小线程池（3个线程）
        ExecutorService executor =
        Executors.newFixedThreadPool(3);

        // 2. 提交 5 个任务
        for (int i = 0; i < 5; i++) {
            int taskNum = i;
            // 提交 Runnable 任务（无返回值）
            executor.submit(() -> {

                System.out.println(Thread.currentThread().getName() + " 执行任务" +
                taskNum);

                try {
                    Thread.sleep(500);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            });
        }

        // 3. 关闭线程池（先执行完已提交的任务，不再接受新任务）
        executor.shutdown();
    }
}

```

示例 2：提交 Callable 任务（有返回值）

java

运行

```

import java.util.concurrent.ExecutorService;

```

```

import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class ThreadPoolCallableDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService executor =
            Executors.newSingleThreadExecutor(); // 单线程池

        // 提交 Callable 任务, 返回 Future 对象
        Future<Integer> future = executor.submit(() -> {
            Thread.sleep(1000);
            return 100 + 200; // 返回值 300
        });

        System.out.println("等待任务执行完成...");
        Integer result = future.get(); // 阻塞获取结果
        System.out.println("任务返回结果: " + result); // 输出 300

        executor.shutdown();
    }
}

```

生产环境推荐：自定义 ThreadPoolExecutor

`Executors` 创建的线程池存在潜在风险（如 `newCachedThreadPool` 无界线程数可能导致 OOM），推荐直接使用 `ThreadPoolExecutor` 自定义参数：

java

运行

```

import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

public class CustomThreadPoolDemo {
    public static void main(String[] args) {

```

```

// 自定义线程池参数
ThreadPoolExecutor executor = new ThreadPoolExecutor(
    2, // 核心线程数（常驻线程）
    5, // 最大线程数
    60, // 空闲线程存活时间
    TimeUnit.SECONDS, // 存活时间单位
    new ArrayBlockingQueue<>(10), // 任务队列（有界，容量
10)
    new ThreadPoolExecutor.AbortPolicy() // 拒绝策略（任
务满时抛出异常）
);

// 提交任务
for (int i = 0; i < 15; i++) {
    int taskNum = i;
    executor.submit(() -> {

System.out.println(Thread.currentThread().getName() + " 执行任务" +
taskNum);

        try {
            Thread.sleep(500);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    });
}

executor.shutdown();
}
}

```

线程池核心优势：

- 线程复用：减少线程创建 / 销毁的开销；
- 统一管理：控制最大线程数，避免资源耗尽；
- 任务队列：缓冲待执行任务，削峰填谷；
- 拒绝策略：任务过多时的优雅处理（如抛异常、丢弃任务）。

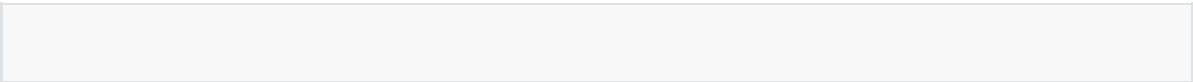
三、线程的生命周期与状态转换

Java 线程有 6 种状态（定义在 `Thread.State` 枚举中），状态转换是多线程编程的核心逻辑，需牢记转换路径：

1. 6 种线程状态

状态名称	说明
NEW	线程已创建但未启动（未调用 <code>start()</code> 方法）
RUNNABLE	线程正在执行（或等待 CPU 调度），包含“运行中”和“就绪”两种状态
BLOCKED	线程等待获取锁（如 <code>synchronized</code> 锁），进入阻塞状态
WAITING	线程无限期等待（如 <code>Object.wait()</code> 、 <code>Thread.join()</code> ），需被其他线程唤醒
TIMED_WAITING	线程限时等待（如 <code>Thread.sleep(1000)</code> 、 <code>Object.wait(1000)</code> ），超时自动唤醒
TERMINATED	线程执行完成（ <code>run()</code> 方法结束）或异常终止

2. 状态转换图（核心路径）



一、先明确：Java 线程的 6 种核心状态 (Thread.State 枚举)

- 1. **NEW (新建态)**：线程对象已创建，但没调用 `start()` 方法（比如 `new Thread()` 后，还没执行 `thread.start()`）；
- 2. **RUNNABLE (可运行态)**：调用 `start()` 后进入，包含两种情况——要么正在 CPU 上执行（运行中），要么在等待 CPU 调度（就绪）；
- 3. **BLOCKED (阻塞态)**：专门等待获取 `synchronized` 锁（比如线程 A 持有锁，线程 B 抢不到就进入 BLOCKED）；
- 4. **WAITING (无限等待态)**：主动放弃 CPU 和锁，无限期等待其他线程唤醒（比如调用 `object.wait()`、`thread.join()` 无参版）；
- 5. **TIMED_WAITING (限时等待态)**：在指定时间内等待，超时会自动唤醒（比如 `Thread.sleep(1000)`、`object.wait(2000)`）；
- 6. **TERMINATED (终止态)**：线程执行完 `run()` 方法，或因未捕获异常终止。

二、核心状态转换路径（按“从创建到终止”逻辑梳理）

1. 启动路径：NEW → RUNNABLE

- 触发条件：调用线程对象的 `start()` 方法（注意：不能直接调用 `run()`，否则只是普通方法调用，不会启动新线程）；
- 示例：`Thread thread = new Thread(); thread.start();` → 线程从 NEW 变成 RUNNABLE。

2. 运行中切换：RUNNABLE ↔ 3 种等待 / 阻塞态

这是最核心的转换，分 3 种场景：

- 场景 1：RUNNABLE → BLOCKED（抢锁失败）
 - 触发：线程竞争 `synchronized` 锁，或等待 `Class` 对象锁（静态同步方法）失败；
 - 转换回 RUNNABLE：持有锁的线程释放锁（退出同步块 / 方法），当前线程抢到锁。
- 场景 2：RUNNABLE → WAITING（无限等待）
 - 触发动作（3 种常见）：
 1. 调用 `Object.wait()`（无参）：必须在 `synchronized` 块内调用，释放锁；
 2. 调用 `Thread.join()`（无参）：等待目标线程执行完毕；
 3. 调用 `LockSupport.park()`：无参版，需其他线程调用 `LockSupport.unpark(thread)` 唤醒；
 - 转换回 RUNNABLE：其他线程执行对应唤醒操作（比如 `object.notify()`、`LockSupport.unpark()`）。
- 场景 3：RUNNABLE → TIMED_WAITING（限时等待）
 - 触发动作（5 种常见）：
 1. `Thread.sleep(long ms)`：不释放锁，仅放弃 CPU；
 2. `Object.wait(long ms)`：释放锁，超时自动唤醒；
 3. `Thread.join(long ms)`：限时等待目标线程；
 4. `LockSupport.parkNanos(long nanos)`：限时阻塞；
 5. `LockSupport.parkUntil(long deadline)`：到指定时间自动唤醒；
 - 转换回 RUNNABLE：要么超时，要么被其他线程唤醒（比如 `notify()`、`unpark()`）。

3. 终止路径：RUNNABLE → TERMINATED

- 触发条件（2 种）：
 1. 线程的 `run()` 方法正常执行完毕；
 2. 线程执行中抛出未捕获的异常（比如 `NullPointerException`），导致终止；
- 注意：线程进入 `TERMINATED` 后，不能再调用 `start()` 重启（会抛 `IllegalThreadStateException`）。

三、一张表总结所有转换关系（清晰无遗漏）

当前状态	可转换到的状态	触发条件
NEW	RUNNABLE	调用 <code>thread.start()</code>
RUNNABLE	BLOCKED	竞争 <code>synchronized</code> 锁失败
RUNNABLE	WAITING	调用 <code>wait()</code> （无参）、 <code>join()</code> （无参）、 <code>LockSupport.park()</code>
RUNNABLE	TIMED_WAITING	调用 <code>sleep(ms)</code> 、 <code>wait(ms)</code> 、 <code>join(ms)</code> 、 <code>parkNanos()</code> 、 <code>parkUntil()</code>
RUNNABLE	TERMINATED	<code>run()</code> 执行完毕，或抛出未捕获异常
BLOCKED	RUNNABLE	抢到 <code>synchronized</code> 锁
WAITING	RUNNABLE	被其他线程唤醒（ <code>notify()</code> 、 <code>unpark()</code> 等）
TIMED_WAITING	RUNNABLE	等待超时，或被其他线程唤醒
TERMINATED	无（最终状态）	无

四、关键易错点提醒

1. `RUNNABLE` 不是“正在运行”：它包含“就绪”和“运行中”，线程是否真的执行，取决于 CPU 调度；
2. `BLOCKED` 只和 `synchronized` 有关：如果是等待 `ReentrantLock` 锁，线程状态是 `WAITING` 或 `TIMED_WAITING`（不是 `BLOCKED`）；
3. `sleep()` 和 `wait()` 的区别：`sleep()` 不释放锁，状态是 `TIMED_WAITING`；`wait()` 释放锁，状态是 `WAITING` / `TIMED_WAITING`；

4. 终止态不可逆转：线程一旦变成 `TERMINATED`，就彻底结束，无法通过任何方法恢复。

生成失败，请重试

3. 关键状态转换场景

- `NEW` → `RUNNABLE`：调用 `thread.start()` 方法；
- `RUNNABLE` → `BLOCKED`：线程竞争 `synchronized` 锁失败；
- `RUNNABLE` → `WAITING`：调用 `object.wait()`（无超时）、`thread.join()`（无超时）；
- `RUNNABLE` → `TIMED_WAITING`：调用 `Thread.sleep(ms)`、`object.wait(ms)`、`thread.join(ms)`；
- `BLOCKED/WAITING/TIMED_WAITING` → `RUNNABLE`：获取锁、被唤醒（`notify()`）或超时；
- `RUNNABLE` → `TERMINATED`：`run()` 方法执行完毕或抛出未捕获异常。

四、线程同步机制（解决线程安全问题）

多线程并发访问共享资源时，会出现线程安全问题（如脏读、重复计算）。Java 提供多种同步机制，核心是“保证共享资源的原子性、可见性、有序性”。

1. 原子性、可见性、有序性（线程安全三要素）

- **原子性**：操作不可分割（如 `i++` 不是原子操作，需拆分为“读 - 改 - 写”三步，可能被线程打断）；
- **可见性**：一个线程修改共享资源后，其他线程能立即看到最新值（避免 CPU 缓存导致的旧值读取）；
- **有序性**：线程执行顺序符合代码逻辑（避免编译器 / CPU 重排序导致的执行混乱）。

2. 同步机制 1：synchronized（隐式锁，推荐基础使用）

`synchronized` 是 Java 内置的隐式锁（监视器锁），自动实现锁的获取和释放，支持修饰方法和代码块，能保证原子性、可见性、有序性。

(1) 修饰代码块 (推荐, 锁粒度更细)

锁定指定对象 (通常是 `this` 或静态对象), 仅代码块内的逻辑同步:

java

运行

```
public class SynchronizedBlockDemo {
    private int count = 0;
    private final Object lock = new Object(); // 锁对象 (推荐使用专用
    对象, 避免锁冲突)

    // 线程安全的自增方法
    public void increment() {
        synchronized (lock) { // 锁定 lock 对象
            count++; // 原子操作 (被锁保护)
        }
    }

    public int getCount() {
        return count;
    }

    // 测试
    public static void main(String[] args) throws
    InterruptedException {
        SynchronizedBlockDemo demo = new SynchronizedBlockDemo();
        ExecutorService executor =
        Executors.newFixedThreadPool(10);

        // 10 个线程各执行 1000 次自增
        for (int i = 0; i < 10; i++) {
            executor.submit(() -> {
                for (int j = 0; j < 1000; j++) {
                    demo.increment();
                }
            })
        }
    }
}
```

```

        });
    }

    executor.shutdown();
    executor.awaitTermination(1, TimeUnit.SECONDS); // 等待所有
线程执行完成

    System.out.println("最终 count 值: " + demo.getCount()); //
输出 10000 (线程安全)
}
}

```

(2) 修饰实例方法（锁定当前对象 `this`）

java

运行

```

// 同步实例方法（锁对象为 this）
public synchronized void increment() {
    count++;
}

```

(3) 修饰静态方法（锁定类对象 `XXX.class`）

java

运行

```
private static int staticCount = 0;

// 同步静态方法（锁对象为 SynchronizedDemo.class）
public static synchronized void staticIncrement() {
    staticCount++;
}
```

synchronized 核心原理：

- 锁的本质是对象头中的“监视器锁（monitor）”，线程获取锁时进入 monitor 的 EntryList 阻塞，获取锁后执行代码，释放锁时唤醒阻塞线程；
- 自动释放锁：无论正常执行完还是抛出异常，JVM 都会自动释放锁（避免死锁风险）；
- 可见性保障：释放锁时会将工作内存的修改同步到主内存，获取锁时会清空工作内存，从主内存加载最新值；
- 有序性保障：禁止重排序（通过内存屏障实现）。

3. 同步机制 2：ReentrantLock（显式锁，功能更强 大）

`ReentrantLock` (JDK 5+) 是 `java.util.concurrent.locks` 包下的显式锁，需手动调用 `lock()` 获取锁、`unlock()` 释放锁，功能比 `synchronized` 更灵活（如公平锁、超时锁、条件变量）。

核心特性：

- 可重入：同一线程可多次获取同一把锁（与 `synchronized` 一致）；
- 公平锁 / 非公平锁：默认非公平锁（效率高），可通过构造函数 `new ReentrantLock(true)` 实现公平锁（按线程等待顺序获取锁）；
- 超时锁：支持 `tryLock(ms, TimeUnit)` 方法，超时未获取锁则返回 `false`（避免无限阻塞）；
- 条件变量：通过 `newCondition()` 获取 `Condition` 对象，支持精准唤醒线程（而非 `notify()` 随机唤醒）。

示例代码（超时锁 + 条件变量）：

java

运行

```

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class ReentrantLockDemo {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notEmpty = lock.newCondition(); // 条件变量：队列非空
    private final Condition notFull = lock.newCondition(); // 条件变量：队列未满足
    private final int[] queue = new int[5];
    private int count = 0; // 队列中元素个数
    private int putIndex = 0; // 放入索引
    private int takeIndex = 0; // 取出索引

    // 放入元素（生产者）
    public void put(int value) throws InterruptedException {
        lock.lock(); // 获取锁
        try {
            // 队列满时，等待 notFull 条件（释放锁，进入等待队列）
            while (count == queue.length) {
                notFull.await(); // 等待，直到队列未满足
            }
            queue[putIndex] = value;
            putIndex = (putIndex + 1) % queue.length;
            count++;
            notEmpty.signal(); // 唤醒等待 notEmpty 条件的线程（消费者）

            System.out.println("生产者放入: " + value + ", 当前队列大小: " + count);
        } finally {
            lock.unlock(); // 释放锁（必须在 finally 中，避免异常导致锁泄漏）
        }
    }

    // 取出元素（消费者）
    public int take() throws InterruptedException {
        lock.lock();

```

```

        try {
            // 队列空时, 等待 notEmpty 条件
            while (count == 0) {
                notEmpty.await();
            }
            int value = queue[takeIndex];
            takeIndex = (takeIndex + 1) % queue.length;
            count--;
            notFull.signal(); // 唤醒等待 notFull 条件的线程 (生产者)
            System.out.println("消费者取出: " + value + ", 当前队列大小: " + count);
            return value;
        } finally {
            lock.unlock();
        }
    }

    // 测试 (生产者-消费者模型)
    public static void main(String[] args) {
        ReentrantLockDemo queue = new ReentrantLockDemo();
        ExecutorService executor =
        Executors.newFixedThreadPool(4);

        // 2 个生产者
        for (int i = 0; i < 2; i++) {
            int producerId = i;
            executor.submit(() -> {
                try {
                    for (int j = 0; j < 5; j++) {
                        queue.put(producerId * 10 + j);
                        Thread.sleep(200);
                    }
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            });
        }

        // 2 个消费者
        for (int i = 0; i < 2; i++) {
            executor.submit(() -> {
                try {
                    for (int j = 0; j < 5; j++) {
                        queue.take();
                        Thread.sleep(300);
                    }
                }
            });
        }
    }

```

```
        }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    });
}

executor.shutdown();
}
}
```

ReentrantLock vs synchronized:

特性	ReentrantLock	synchronized
锁的获取 / 释放	手动 (lock ()/unlock ())	自动 (进入 / 退出同步块)
公平锁支持	支持 (构造函数指定)	不支持 (仅非公平锁)
超时锁支持	支持 (tryLock (ms))	不支持 (无限阻塞)
条件变量	支持 (Condition, 精准唤醒)	支持 (wait ()/notify (), 随机唤醒)
锁状态查询	支持 (isLocked () 等方法)	不支持
性能	高并发下性能更优	低并发下简单高效
推荐场景	高并发、需要灵活控制锁的场景	普通同步场景 (简单易用)

4. 同步机制 3: volatile (轻量级同步, 仅保证可见性和有序性)

volatile 是轻量级同步关键字, 用于修饰变量, 不能保证原子性, 仅能保证可见性和有序性, 适用于“单线程写、多线程读”的场景 (如状态标记位)。

核心作用:

- 可见性: 一个线程修改 volatile 变量后, 会立即同步到主内存, 其他线程读取时会从主内存加载最新值 (避免 CPU 缓存导致的旧值读取);
- 有序性: 禁止 volatile 变量前后的指令重排序 (通过内存屏障实现);
- 不保证原子性: 如 volatile int i = 0; i++ 仍存在线程安全问题 (需结合原子类或锁)。

示例代码（状态标记位）：

java

运行

```
public class volatileDemo {
    private volatile boolean isStop = false; // volatile 修饰状态标
    记位

    public void run() {
        System.out.println("线程启动，开始执行...");
        while (!isStop) {
            // 业务逻辑
        }
        System.out.println("线程停止，执行结束...");
    }

    public void stop() {
        isStop = true; // 修改标记位，线程会立即看到最新值
    }

    public static void main(String[] args) throws
    InterruptedException {
        volatileDemo demo = new volatileDemo();
        Thread thread = new Thread(demo::run);
        thread.start();

        Thread.sleep(1000); // 让线程执行 1 秒
        demo.stop(); // 停止线程
    }
}
```

volatile 适用场景：

- 状态标记位（如是否停止线程、是否初始化完成）；
- 单例模式的双重检查锁定（DCL）优化（避免重排序导致的空指针）：

java

运行

```
public class Singleton {  
    // volatile 禁止重排序，确保 instance 初始化完成后再赋值  
    private static volatile Singleton instance;  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null) { // 第一次检查（无锁，提高效率）  
            synchronized (Singleton.class) { // 加锁  
                if (instance == null) { // 第二次检查（避免多线程竞  
争创建多个实例）  
                    instance = new Singleton(); // 禁止重排序  
                }  
            }  
        }  
        return instance;  
    }  
}
```

5. 同步机制 4：原子类（AtomicXXX，无锁同步）

`java.util.concurrent.atomic` 包下的原子类（如 `AtomicInteger`、`AtomicLong`、`AtomicReference`）基于 CAS（Compare And Swap）操作实现无锁同步，保证原子性，性能优于锁（避免上下文切换）。

CAS 原理：

CAS 是一种乐观锁机制，核心是“比较并替换”：

1. 假设变量当前值为 `expected`，期望修改为 `newValue`；
2. 若变量实际值与 `expected` 一致，则修改为 `newValue`，返回 `true`；
3. 若不一致，则不修改，返回 `false`（可重试或放弃）；
4. 底层通过 CPU 指令 `cmpxchg` 实现，保证原子性。

示例代码（原子类自增）：

java

运行

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicDemo {
    private final AtomicInteger atomicCount = new
AtomicInteger(0); // 原子类

    // 线程安全的自增（无锁）
    public void increment() {
        atomicCount.incrementAndGet(); // 原子自增（相当于 count++）
    }

    public int getCount() {
        return atomicCount.get();
    }

    public static void main(String[] args) throws
InterruptedException {
        AtomicDemo demo = new AtomicDemo();
    }
}
```

```

        ExecutorService executor =
Executors.newFixedThreadPool(10);

        for (int i = 0; i < 10; i++) {
            executor.submit(() -> {
                for (int j = 0; j < 1000; j++) {
                    demo.increment();
                }
            });
        }

        executor.shutdown();
        executor.awaitTermination(1, TimeUnit.SECONDS);

        System.out.println("最终 count 值: " + demo.getCount()); //
输出 10000 (线程安全)
    }
}

```

常用原子类：

- 基本类型原子类： `AtomicInteger`、`AtomicLong`、`AtomicBoolean`；
- 引用类型原子类： `AtomicReference`（原子修改对象引用）、`AtomicStampedReference`（解决 CAS ABA 问题）；
- 数组类型原子类： `AtomicIntegerArray`、`AtomicLongArray`；
- 字段更新器： `AtomicIntegerFieldUpdater`（原子修改对象的非原子字段）。

五、Java 并发工具类（提升开发效率）

JDK 提供的 `java.util.concurrent` 包包含多种高效并发工具类，避免重复造轮子，核心常用类如下：

1. CountDownLatch（倒计时器）

用于等待多个线程完成任务后，主线程再继续执行（如：主线程等待 10 个子线程全部执行完后汇总结果）。

核心方法：

- `CountDownLatch(int count)`：初始化倒计时次数；
- `countDown()`：线程完成任务后调用，倒计时次数减 1；
- `await()`：主线程调用，阻塞直到倒计时为 0。

示例代码：

java

运行

```
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class CountDownLatchDemo {
    public static void main(String[] args) throws
InterruptedException {
        int threadNum = 5;
        CountDownLatch latch = new CountDownLatch(threadNum); //
倒计时 5 次

        ExecutorService executor =
Executors.newFixedThreadPool(threadNum);
        for (int i = 0; i < threadNum; i++) {
            int taskId = i;
            executor.submit(() -> {
                try {
                    System.out.println("线程" + taskId + "开始执行任
务");

                    Thread.sleep(1000); // 模拟任务执行
                    System.out.println("线程" + taskId + "完成任
务");

                } catch (InterruptedException e) {
                    e.printStackTrace();
                } finally {
                    latch.countDown(); // 倒计时减 1（必须在 finally
中，确保执行）
                }
            });
        }
    }
}
```

```

        System.out.println("主线程等待所有子线程完成...");
        latch.await(); // 阻塞直到所有子线程完成
        System.out.println("所有子线程执行完成，主线程继续执行");

        executor.shutdown();
    }
}

```

2. CyclicBarrier (循环栅栏)

让多个线程在某个“栅栏”处等待，直到所有线程都到达栅栏后，再一起继续执行（可循环使用，如：多线程分阶段执行任务，每个阶段等待所有线程就绪）。

核心方法：

- `CyclicBarrier(int parties)`：初始化参与线程数；
- `CyclicBarrier(int parties, Runnable barrierAction)`：所有线程到达后，先执行 `barrierAction`（如汇总结果）；
- `await()`：线程到达栅栏后调用，阻塞直到所有线程都到达。

示例代码（多线程分阶段执行）：

java

运行

```

import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class CyclicBarrierDemo {
    public static void main(String[] args) {
        int threadNum = 3;
        // 所有线程到达后，执行栅栏动作（打印提示）
        CyclicBarrier barrier = new CyclicBarrier(threadNum, () ->
    {

```

```

        System.out.println("=== 所有线程已到达栅栏，开始下一阶段
===");
    });

    ExecutorService executor =
Executors.newFixedThreadPool(threadNum);
    for (int i = 0; i < threadNum; i++) {
        int threadId = i;
        executor.submit(() -> {
            try {
                // 第一阶段任务
                System.out.println("线程" + threadId + "执行第一
阶段任务");

                Thread.sleep((int) (Math.random() * 1000));
                System.out.println("线程" + threadId + "到达第一
阶段栅栏");

                barrier.await(); // 等待其他线程

                // 第二阶段任务
                System.out.println("线程" + threadId + "执行第二
阶段任务");

                Thread.sleep((int) (Math.random() * 1000));
                System.out.println("线程" + threadId + "到达第二
阶段栅栏");

                barrier.await(); // 等待其他线程

            } catch (Exception e) {
                e.printStackTrace();
            }
        });
    }

    executor.shutdown();
}
}

```

3. Semaphore (信号量)

用于控制同时访问某个资源的线程数（如：限制最大并发连接数、限流），核心是“许可”机制：线程获取许可后才能访问资源，访问完成后释放许可。

核心方法：

- `Semaphore(int permits)`：初始化许可数（最大并发数）；
- `acquire()`：获取许可（无许可则阻塞）；
- `tryAcquire()`：尝试获取许可（无许可则返回 false，不阻塞）；
- `release()`：释放许可。

示例代码（限流场景）：

java

运行

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Semaphore;

public class SemaphoreDemo {
    public static void main(String[] args) {
        int maxConcurrent = 2; // 最大并发数（许可数）
        Semaphore semaphore = new Semaphore(maxConcurrent);

        ExecutorService executor =
            Executors.newFixedThreadPool(5);
        for (int i = 0; i < 5; i++) {
            int taskId = i;
            executor.submit(() -> {
                try {
                    semaphore.acquire(); // 获取许可（最多 2 个线程同
时获取）

                    System.out.println("任务" + taskId + "获取许可，
开始执行");

                    Thread.sleep(1000); // 模拟资源访问
                    System.out.println("任务" + taskId + "执行完成，
释放许可");

                } catch (InterruptedException e) {
```

```

        e.printStackTrace();
    } finally {
        semaphore.release(); // 释放许可（必须在 finally
中）
    }
});
}

executor.shutdown();
}
}

```

4. ConcurrentHashMap（并发集合）

`HashMap` 线程不安全，`Hashtable` 效率低（全表锁），`ConcurrentHashMap` 是线程安全的哈希表，采用“分段锁”（JDK 7）或“CAS + synchronized”（JDK 8+）实现，支持高并发读写。

核心特性：

- 线程安全：读写操作均为线程安全，无需额外加锁；
- 高效并发：读操作无锁（JDK 8+），写操作仅锁定当前桶（而非全表）；
- 支持原子操作：提供 `putIfAbsent()`、`compute()` 等原子方法。

示例代码：

java

运行

```

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ConcurrentHashMapDemo {

```

```

    public static void main(String[] args) throws
InterruptedException {
        Map<String, Integer> map = new ConcurrentHashMap<>();
        ExecutorService executor =
Executors.newFixedThreadPool(10);

        // 10 个线程同时向 map 中 put 数据
        for (int i = 0; i < 10; i++) {
            int num = i;
            executor.submit(() -> {
                map.put("key" + num, num);

                System.out.println(Thread.currentThread().getName() + " put: key"
+ num + "=" + num);
            });
        }

        executor.shutdown();
        executor.awaitTermination(1, TimeUnit.SECONDS);

        System.out.println("map 大小: " + map.size()); // 输出 10
        (线程安全)
        System.out.println("map 内容: " + map);
    }
}

```

其他常用并发集合：

- `CopyOnWriteArrayList`：读多写少场景的线程安全列表（写操作复制全表，读操作无锁）；
- `BlockingQueue`：阻塞队列（如 `ArrayBlockingQueue`、`LinkedBlockingQueue`），用于生产者 - 消费者模型；
- `ConcurrentLinkedQueue`：无锁并发队列（CAS 实现），高并发场景下性能优。

六、多线程常见问题与解决方案

1. 死锁 (Deadlock)

现象：

多个线程互相持有对方需要的锁，无限期阻塞（如线程 A 持有锁 1，等待锁 2；线程 B 持有锁 2，等待锁 1）。

死锁产生条件（缺一不可）：

1. 互斥条件：资源只能被一个线程持有；
2. 请求与保持条件：线程持有一个资源，同时请求另一个资源；
3. 不可剥夺条件：资源不能被强制剥夺；
4. 循环等待条件：线程间形成循环等待资源的关系。

示例代码（死锁场景）：

java

运行

```
public class DeadlockDemo {
    private static final Object lock1 = new Object();
    private static final Object lock2 = new Object();

    public static void main(String[] args) {
        // 线程 1: 持有 lock1, 请求 lock2
        new Thread(() -> {
            synchronized (lock1) {
                System.out.println("线程1持有 lock1, 请求 lock2");
                try {
                    Thread.sleep(100);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                synchronized (lock2) {
                    System.out.println("线程1获取 lock2");
                }
            }
        }, "线程1").start();
    }
}
```

```

// 线程 2: 持有 lock2, 请求 lock1
new Thread(() -> {
    synchronized (lock2) {
        System.out.println("线程2持有 lock2, 请求 lock1");
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        synchronized (lock1) {
            System.out.println("线程2获取 lock1");
        }
    }
}, "线程2").start();
}
}

```

解决方案:

- 破坏循环等待条件: 统一锁的获取顺序 (如线程都先获取 lock1, 再获取 lock2) ;
- 破坏请求与保持条件: 一次性获取所有需要的锁;
- 超时锁: 使用 `ReentrantLock.tryLock(ms)`, 超时未获取锁则释放已持有锁;
- 监控死锁: 使用 `jstack <pid>` 命令查看线程状态, 定位死锁线程和锁资源。

2. 线程泄漏 (Thread Leak)

现象:

线程创建后未正常终止, 长期占用资源 (如线程池核心线程持有任务引用, 任务未结束导致线程无法回收) 。

常见原因:

- 线程池任务中存在无限循环 (如 `while (true)` 未加退出条件) ;
- 线程阻塞在 `Object.wait()`、`Thread.join()` 等方法, 未被唤醒;
- 自定义线程未设置为守护线程, 主线程退出后仍在运行。

解决方案：

- 线程任务中添加退出条件（如通过 `volatile` 标记位控制循环）；
- 避免无限期阻塞，使用超时机制（如 `wait(ms)`、`tryLock(ms)`）；
- 合理设置线程池参数（如核心线程数、空闲线程存活时间）；
- 使用守护线程（`thread.setDaemon(true)`），主线程退出后自动终止。

3. 上下文切换过多

现象：

线程数过多（远超 CPU 核心数），导致 CPU 频繁切换线程，上下文切换开销占比高，程序吞吐量下降。

解决方案：

- 控制线程数：线程池最大线程数建议设置为 CPU 核心数 * 2（IO 密集型）或 CPU 核心数（CPU 密集型）；
- 使用无锁并发：如原子类、`ConcurrentHashMap`，减少锁竞争导致的上下文切换；
- 批量处理任务：减少任务提交次数，降低线程切换频率；
- 避免线程阻塞：减少 IO 操作、锁等待等阻塞场景（如使用异步 IO）。

七、多线程性能优化建议（生产实战）

1. **优先使用线程池，而非直接创建线程**：线程池复用线程，减少创建 / 销毁开销，统一管理线程；
2. **减少锁竞争**
 - ：
 - 锁粒度最小化（如 `synchronized` 修饰代码块而非方法）；
 - 读多写少场景使用 `CopyOnWriteArrayList`、`ConcurrentHashMap`（无锁读）；
 - 高并发计数场景使用 `LongAdder`（比 `AtomicLong` 性能更优）；
3. **避免线程阻塞**
 - ：
 - IO 操作（如数据库查询、网络请求）使用异步方式（如 `CompletableFuture`）；
 - 避免无限期阻塞，使用超时机制；

4. **合理设置线程优先级**：默认优先级即可，避免过度依赖 `setPriority()`（不同 OS 支持不同）；
5. **使用本地变量**：局部变量存储在线程栈中，无线程安全问题，无需同步，性能更优；
6. **避免创建过多临时对象**：多线程场景下频繁创建对象会增加 GC 压力，建议复用对象（如对象池）；
7. 选择合适的并发工具
：
 - 计数 / 统计：原子类（`AtomicXXX`、`LongAdder`）；
 - 高并发读写：`ConcurrentHashMap`；
 - 生产者 - 消费者：`BlockingQueue`；
 - 限流：`Semaphore`。

八、总结

Java 多线程是提升程序并发能力的核心，核心要点可归纳为：

1. **线程创建**：生产环境首选线程池（`ThreadPoolExecutor`），避免直接创建线程；
2. **线程安全**：通过 `synchronized`、`ReentrantLock` 保证原子性，`volatile` 保证可见性 / 有序性，原子类实现无锁同步；
3. **并发工具**：`CountDownLatch`、`CyclicBarrier`、`Semaphore` 等工具类简化多线程协作；
4. **问题排查**：死锁用 `jstack` 定位，内存泄漏用 `jmap+MAT` 分析，性能问题用 `Arthas` 追踪；
5. **性能优化**：减少锁竞争、避免线程阻塞、控制线程数，优先使用无锁并发。

111

1 线程基础：进程与线程、并发与并行

表格

复制

概念	说明	比喻
进程	OS 资源分配基本单位，拥有独立地址空间	一个公司
线程	CPU 调度基本单位，共享进程内存	公司里的员工
并发	单核交替执行（时间片）	一人多岗
并行	多核真正同时执行	多人协同

2 创建线程的 4 种官方姿势（JDK21 仍有效）

表格

复制

方式	语法	特点	场景
继承 Thread	<code>class T extends Thread</code>	简单，单继承局限	快速测试/教学
实现 Runnable	<code>new Thread(() -> {...})</code>	解耦，可复用	绝大多数业务
实现 Callable	<code>Future f = executor.submit(callable)</code>	有返回值，可抛异常	需要结果的任务
线程池	<code>Executors.newFixedThreadPool(n)</code>	复用线程，统一治理	生产唯一入口

结论：生产环境永远用线程池；前三种只出现在面试题。

3 线程生命周期（6 状态模型）

NEW → RUNNABLE → BLOCKED → WAITING → TIMED_WAITING → TERMINATED

- NEW：刚 `new Thread()`
- RUNNABLE：`start()` 后，就绪 + 运行 统称
- BLOCKED：抢 `synchronized` 失败
- WAITING：无限期 等 `Object.wait()` / `LockSupport.park()`

- **TIMED_WAITING**: 带超时 等 `Thread.sleep(long)` / `join(long)`
- **TERMINATED**: `run()` 结束

命令行看状态：

bash

复制

```
jstack <pid> | grep -A 5 java.lang.Thread.State
```

4 线程池：生产环境唯一入口（面试 + 实战核心）

4.1 核心 7 参数（ThreadPoolExecutor）

java

复制

```
public ThreadPoolExecutor(  
    int corePoolSize,           // 核心线程（常驻）  
    int maximumPoolSize,       // 最大线程  
    long keepAliveTime,        // 空闲线程存活  
    TimeUnit unit,             // 时间单位  
    BlockingQueue<Runnable> workQueue, // 任务队列  
    ThreadFactory threadFactory, // 命名、优先级  
    RejectedExecutionHandler handler // 拒绝策略  
)
```

4.2 预定义 4 池（生产慎用，队列无界易 OOM）

表格

复制

方法	队列	风险
<code>newFixedThreadPool</code>	<code>LinkedBlockingQueue</code> 无界	任务堆积 OOM
<code>newCachedThreadPool</code>	<code>SynchronousQueue</code>	线程数 MAX → CPU 打满
<code>newSingleThreadExecutor</code>	无界	同 Fixed

方法	队列	风险
<code>newScheduledThreadPool</code>	<code>DelayedWorkQueue</code>	同 <code>Cached</code>

结论：必须自定义 `ThreadPoolExecutor`，给出界队列 + 拒绝策略。

4.3 生产级线程池模板（可直接复制）

java

复制

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(
    8,                      // core = NCPU
    16,                     // max = NCPU*2
    60L, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(1000), // 有界队列
    new ThreadFactoryBuilder().setNameFormat("biz-%d").build(),
    new ThreadPoolExecutor.CallerRunsPolicy() // 调用者线程兜底
);
```

关闭优雅：

java

复制

```
pool.shutdown(); // 不再接受新任务
if (!pool.awaitTermination(30, TimeUnit.SECONDS))
    pool.shutdownNow(); // 强制中断
```

5 并发工具包（JUC）速查

表格

复制

工具类	作用	典型场景
<code>CountDownLatch</code>	等待多个线程完成	启动前加载多份配置
<code>CyclicBarrier</code>	循环汇合点	多阶段并行计算
<code>Semaphore</code>	限流	接口 QPS 控制
<code>Exchanger</code>	两线程交换数据	生产者-消费者单对单

工具类	作用	典型场景
CompletableFuture	异步编排	多服务并行调用
BlockingQueue	生产者-消费者	线程池底层队列

6 线程安全：synchronized 进化史

表格

复制

版本	锁状态	特点
JDK6 之前	重量级锁	用户态→内核态，成本高
JDK6+	偏向锁→轻量级锁→重量级锁	无竞争偏向，CAS 轻量
JDK15	偏向锁默认关闭	撤销代价 > 收益

结论：短临界区直接用 `synchronized`，长/复杂用 `ReentrantLock`（可中断、公平、条件队列）。

7 虚拟线程（Project Loom，JDK21 正式）

- 创建方式：

java

复制

```
Thread vtThread = Thread.ofVirtual().name("vt-").start(() -> {
    // 业务逻辑
});
```

- 特点：JVM 级用户态线程，1 MB 堆栈→几百字节，百万并发单机
- 适用：IO 密集型（网关、网关、RPC 聚合）
- 不适用：CPU 密集型（计算、加密）

搭配 结构化并发（`StructuredTaskScope`）实现“线程超时/取消”自动管理。

8 性能调优 6 板斧

表格

复制

问题	工具	命令/参数
CPU 飙高	<code>top -H + jstack</code>	定位 TID → 十六进制 → 栈
死锁	<code>jcmd Thread.print</code>	自动输出死锁链
线程数暴涨	<code>jstack grep java.lang.Thread.State wc -l</code>	看 RUNNABLE/WAITING 比例
阻塞	<code>jconsole</code> → 线程页	看 BLOCKED 柱状图
池耗尽	<code>ThreadPoolExecutor#getActiveCount()</code>	监控活跃线程
虚拟线程	<code>jfr</code> 事件 <code>jdk.VirtualThreadStart</code>	JMC 可视化

9 线上案例 1：网关 5 w QPS，线程池打满

现象：RT 暴涨 → 线程池队列拒绝

定位：

bash

复制

```
jstack <pid> | grep "pool-.*-thread" | wc -l    # 仅 200
```

解决：

- `core=max=CPU×2`（避免伸缩）
 - 队列换 `LinkedBlockingQueue(5000)`（有界）
 - 拒绝策略 `CallerRuns`（兜底）
- 结果：RT 恢复 99.9 %。

10 线上案例 2：虚拟线程压测（百万并发）

代码：

java

复制

```
try (var scope = new StructuredTaskScope<Integer>()) {
    for (int i = 0; i < 1_000_000; i++) {
        scope.fork(() -> callRpc(i));
    }
    scope.join();
}
```

结果：16 GB 内存，1 M 并发，P99 20 ms。

11 脑图速记（面试 2 分钟）

复制

Java 多线程

- ├ 创建：Runnable > Callable > 线程池
- ├ 状态：NEW → RUNNABLE → BLOCKED/WAITING → TERMINATED
- ├ 池：7 参数、4 预置池勿用、自定义有界队列 + 拒绝策略
- ├ 安全：synchronized 锁升级、JUC 工具包
- └ 虚拟线程：IO 密集、百万并发、StructuredTaskScope

12 小结

掌握「创建方式 → 生命周期 → 线程池 7 参数 → JUC 工具 → 虚拟线程 → 观测与调优」六件套，面试 2 分钟讲完、上线 5 分钟定位 线程泄露/死锁/池打满 问题。祝学习顺利，若对 虚拟线程调度器、StructuredTaskScope、JUC StampedLock 等想继续深挖，随时再来问！

一、线程基础概念

1.1 什么是线程？

- **进程 (Process)**：操作系统资源分配的基本单位；
- **线程 (Thread)**：进程内执行单元，CPU 调度的基本单位；
- **Java 线程**：JVM 管理的 OS 线程（1:1 映射）。

1.2 线程状态 (Thread.State)

```
public enum State {  
    NEW,                // 新建，未启动  
    RUNNABLE,           // 运行或可运行  
    BLOCKED,            // 阻塞（等待锁）  
    WAITING,            // 等待（Object.wait(), Thread.join()）  
    TIMED_WAITING,      // 计时等待（sleep, wait(timeout)）  
    TERMINATED          // 终止  
}
```

1.3 线程 vs 进程

对比项	线程	进程
资源开销	小（共享进程内存）	大（独立内存空间）
通信方式	共享内存	IPC（管道、消息队列等）
创建销毁	快	慢
安全性	一个线程出错影响整个进程	隔离性好

二、创建线程的四种方式

2.1 继承 Thread 类

```
public class MyThread extends Thread {  
    @Override  
    public void run() {  
        System.out.println("Thread running: " +  
Thread.currentThread().getName());  
    }  
  
    public static void main(String[] args) {
```

```
        MyThread t = new MyThread();
        t.start(); // 启动线程，不能直接调用 run()
    }
}
```

2.3 实现 Callable 接口（有返回值）

```
import java.util.concurrent.*;

public class MyCallable implements Callable<String> {
    @Override
    public String call() throws Exception {
        Thread.sleep(1000);
        return "Task completed!";
    }

    public static void main(String[] args) throws
        ExecutionException, InterruptedException {
        ExecutorService executor =
            Executors.newSingleThreadExecutor();
        Future<String> future = executor.submit(new MyCallable());

        System.out.println("Task result: " + future.get()); // 阻塞等待结果
        executor.shutdown();
    }
}
```

三、线程基本操作

3.1 线程生命周期控制

```
public class ThreadControl {
    public static void main(String[] args) throws
        InterruptedException {
        Thread t = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                System.out.println("Running: " + i);
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    System.out.println("Thread interrupted");
                    return; // 退出线程
                }
            }
        });
    }
}
```

```

        }
    });

    t.start();
    Thread.sleep(3000);
    t.interrupt(); // 中断线程
    t.join(); // 等待线程结束
}
}

```

3.2 线程优先级

```

Thread t = new Thread(() -> {
    // 优先级: 1~10, Thread.MIN_PRIORITY~MAX_PRIORITY
    System.out.println("Priority: " +
        Thread.currentThread().getPriority());
});
t.setPriority(Thread.MAX_PRIORITY);

```

3.3 守护线程 (Daemon)

```

Thread daemon = new Thread(() -> {
    while (true) {
        System.out.println("Daemon running...");
        try { Thread.sleep(1000); } catch (InterruptedException e)
        { break; }
    }
});
daemon.setDaemon(true); // JVM 退出时自动结束
daemon.start();

```

四、并发三大问题

4.1 可见性问题

```

public class visibilityProblem {
    private static boolean flag = false;

    public static void main(String[] args) throws
        InterruptedException {
        new Thread(() -> {
            while (!flag) { } // 可能永远不退出
            System.out.println("Flag is true!");
        });
    }
}

```

```

        }).start();

        Thread.sleep(1000);
        flag = true; // 主线程修改
    }
}

```

4.2 原子性问题

```

public class AtomicityProblem {
    private static int count = 0;

    public static void main(String[] args) throws
InterruptedException {
        Thread[] threads = new Thread[10];
        for (int i = 0; i < 10; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++) {
                    count++; // 非原子操作
                }
            });
        }

        for (Thread t : threads) t.start();
        for (Thread t : threads) t.join();

        System.out.println("Expected: 10000, Actual: " + count);
        // 结果 < 10000
    }
}

```

4.3 有序性问题

```

public class OrderingProblem {
    private static int x = 0, y = 0;
    private static int a = 0, b = 0;

    public static void main(String[] args) throws
InterruptedException {
        int count = 0;
        for (int i = 0; i < 100000; i++) {
            x = y = a = b = 0; // 重置

            Thread t1 = new Thread(() -> {

```

```

        a = 1; x = b;
    });
    Thread t2 = new Thread(() -> {
        b = 1; y = a;
    });

    t1.start(); t2.start();
    t1.join(); t2.join();

    if (x == 0 && y == 0) count++; // 可能发生!
}
System.out.println("x=0,y=0 occurred " + count + "
times");
}
}

```

五、synchronized 关键字详解

5.1 synchronized 的三种用法

```

public class SynchronizedDemo {
    private final Object lock = new Object();

    // 1. 修饰实例方法（锁当前实例）
    public synchronized void method1() {
        System.out.println("Instance method locked: " + this);
    }

    // 2. 修饰静态方法（锁当前类）
    public static synchronized void method2() {
        System.out.println("Static method locked: " +
SynchronizedDemo.class);
    }

    // 3. 修饰代码块（锁指定对象）
    public void method3() {
        synchronized (lock) {
            System.out.println("Block locked: " + lock);
        }
    }
}

```

5.2 synchronized 的实现原理

- **JVM 层面**: 通过 `monitorenter` / `monitorexit` 指令实现;
- **锁升级**: 无锁 → 偏向锁 → 轻量级锁 → 重量级锁 (自适应);
- **性能优化**: JVM 自动优化, 如锁消除、锁粗化。

5.3 死锁示例与避免

```
public class DeadlockDemo {
    private static final Object lock1 = new Object();
    private static final Object lock2 = new Object();

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            synchronized (lock1) {
                System.out.println("Thread 1: lock1 acquired");
                try { Thread.sleep(100); } catch
(InterruptedOperationException e) { }
                synchronized (lock2) { // 等待 lock2
                    System.out.println("Thread 1: lock2
acquired");
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (lock2) {
                System.out.println("Thread 2: lock2 acquired");
                try { Thread.sleep(100); } catch
(InterruptedOperationException e) { }
                synchronized (lock1) { // 等待 lock1
                    System.out.println("Thread 2: lock1
acquired");
                }
            }
        });

        t1.start(); t2.start();
    }
}
```

避免死锁:

1. **固定锁顺序**: 所有线程按相同顺序获取锁;

2. 超时机制: `tryLock(timeout);`
3. 死锁检测: `ThreadMXBean.findDeadlockedThreads()`。

六、JUC 并发包详解

6.1 ReentrantLock (可重入锁)

```
import java.util.concurrent.locks.ReentrantLock;

public class ReentrantLockDemo {
    private final ReentrantLock lock = new ReentrantLock();
    private int count = 0;

    public void increment() {
        lock.lock();
        try {
            count++;
        } finally {
            lock.unlock(); // 必须在 finally 中释放
        }
    }

    // 尝试获取锁（非阻塞）
    public boolean tryIncrement() {
        if (lock.tryLock()) {
            try {
                count++;
                return true;
            } finally {
                lock.unlock();
            }
        }
        return false;
    }

    // 可中断的锁获取
    public void interruptibleIncrement() throws
    InterruptedException {
        lock.lockInterruptibly();
        try {
            count++;
        } finally {
            lock.unlock();
        }
    }
}
```

```
}  
}
```

6.2 ReadWriteLock (读写锁)

```
import java.util.concurrent.locks.ReadWriteLock;  
import java.util.concurrent.locks.ReentrantReadWriteLock;  
  
public class ReadWriteLockDemo {  
    private final ReadWriteLock rwLock = new  
ReentrantReadWriteLock();  
    private final Map<String, String> cache = new HashMap<>();  
  
    public String get(String key) {  
        rwLock.readLock().lock();  
        try {  
            return cache.get(key);  
        } finally {  
            rwLock.readLock().unlock();  
        }  
    }  
  
    public void put(String key, String value) {  
        rwLock.writeLock().lock();  
        try {  
            cache.put(key, value);  
        } finally {  
            rwLock.writeLock().unlock();  
        }  
    }  
}
```

6.3 CountdownLatch (倒计时门闩)

```
import java.util.concurrent.CountDownLatch;  
  
public class CountdownLatchDemo {  
    public static void main(String[] args) throws  
InterruptedException {  
        int threadCount = 3;  
        CountdownLatch latch = new CountdownLatch(threadCount);  
  
        for (int i = 0; i < threadCount; i++) {  
            final int taskId = i;
```

```

        new Thread(() -> {
            System.out.println("Task " + taskId + "
starting...");
            try { Thread.sleep(1000); } catch
(InterruptedException e) { }
            System.out.println("Task " + taskId + "
completed");
            latch.countDown(); // 计数器减1
        }).start();
    }

    System.out.println("waiting for all tasks to
complete...");
    latch.await(); // 等待计数器归零
    System.out.println("All tasks completed!");
}
}

```

6.4 CyclicBarrier (循环屏障)

```

import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;

public class CyclicBarrierDemo {
    public static void main(String[] args) {
        int threadCount = 3;
        CyclicBarrier barrier = new CyclicBarrier(threadCount, ()
-> {
            System.out.println("All threads reached the
barrier!");
        });

        for (int i = 0; i < threadCount; i++) {
            final int workerId = i;
            new Thread(() -> {
                try {
                    System.out.println("worker " + workerId + "
doing work...");
                    Thread.sleep(1000);
                    System.out.println("worker " + workerId + "
waiting at barrier");
                    barrier.await(); // 等待其他线程
                    System.out.println("worker " + workerId + "
passed barrier");
                } catch (InterruptedException e) { }
            }).start();
        }
    }
}

```

```

        } catch (InterruptedException |
BrokenBarrierException e) {
            e.printStackTrace();
        }
    }).start();
}
}
}

```

6.5 Semaphore (信号量)

```

import java.util.concurrent.Semaphore;

public class SemaphoreDemo {
    private static final Semaphore semaphore = new Semaphore(2);
    // 最多2个线程同时访问

    public static void main(String[] args) {
        for (int i = 0; i < 5; i++) {
            final int clientId = i;
            new Thread(() -> {
                try {
                    semaphore.acquire(); // 获取许可
                    System.out.println("Client " + clientId + "
accessing resource");
                    Thread.sleep(2000); // 模拟资源使用
                    System.out.println("Client " + clientId + "
releasing resource");
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                } finally {
                    semaphore.release(); // 释放许可
                }
            }).start();
        }
    }
}

```

七、并发集合类

7.1 ConcurrentHashMap (高性能并发 Map)

```
import java.util.concurrent.ConcurrentHashMap;

public class ConcurrentHashMapDemo {
    private static final ConcurrentHashMap<String, String> map =
new ConcurrentHashMap<>();

    public static void main(String[] args) throws
InterruptedException {
        // 线程安全的 put/get
        Thread writer = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                map.put("key" + i, "value" + i);
            }
        });

        Thread reader = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                String value = map.get("key" + i);
                if (value != null) {
                    System.out.println("Read: " + value);
                }
            }
        });

        writer.start(); reader.start();
        writer.join(); reader.join();
    }
}
```

7.2 BlockingQueue (阻塞队列)

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;

public class ProducerConsumerDemo {
    private static final BlockingQueue<String> queue = new
ArrayBlockingQueue<>(10);

    public static void main(String[] args) {
        // 生产者
```

```

Thread producer = new Thread(() -> {
    for (int i = 0; i < 20; i++) {
        try {
            String item = "Item-" + i;
            queue.put(item); // 队列满时阻塞
            System.out.println("Produced: " + item);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
});

// 消费者
Thread consumer = new Thread(() -> {
    for (int i = 0; i < 20; i++) {
        try {
            String item = queue.take(); // 队列空时阻塞
            System.out.println("Consumed: " + item);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
});

producer.start(); consumer.start();
}
}

```

八、原子类 (java.util.concurrent.atomic)

8.1 基本原子类

```

import java.util.concurrent.atomic.AtomicInteger;

public class AtomicDemo {
    private static final AtomicInteger atomicInt = new
    AtomicInteger(0);

    public static void main(String[] args) throws
    InterruptedException {
        Thread[] threads = new Thread[10];
        for (int i = 0; i < 10; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++) {

```

```

        atomicInt.incrementAndGet(); // 原子操作
    }
});
}

for (Thread t : threads) t.start();
for (Thread t : threads) t.join();

System.out.println("Atomic count: " + atomicInt.get()); //
10000
}
}

```

8.2 AtomicReference (原子引用)

```

import java.util.concurrent.atomic.AtomicReference;

public class AtomicReferenceDemo {
    private static final AtomicReference<String> ref = new
AtomicReference<>("initial");

    public static void main(String[] args) throws
InterruptedException {
        Thread updater = new Thread(() -> {
            String current = ref.get();
            ref.compareAndSet(current, "updated"); // CAS 操作
        });

        updater.start();
        updater.join();
        System.out.println("Value: " + ref.get());
    }
}

```

九、线程池详解

9.1 线程池创建与配置

```

import java.util.concurrent.*;

public class ThreadPoolDemo {
    public static void main(String[] args) {

```

```

// 自定义线程池（推荐）
ThreadPoolExecutor executor = new ThreadPoolExecutor(
    2, // 核心线程数
    4, // 最大线程数
    60L, // 空闲线程存活时间
    TimeUnit.SECONDS, // 时间单位
    new LinkedBlockingQueue<>(100), // 工作队列
    Thread::new, // 线程工厂
    new ThreadPoolExecutor.CallerRunsPolicy() // 拒绝策略
);

// 提交任务
for (int i = 0; i < 10; i++) {
    final int taskId = i;
    executor.submit(() -> {
        System.out.println("Task " + taskId + " executed
by " +
            Thread.currentThread().getName());
        try { Thread.sleep(1000); } catch
(InterruptedException e) { }
    });
}

executor.shutdown(); // 不接受新任务，等待现有任务完成
}
}

```

9.2 Executors 工厂方法（谨慎使用）

```

// 1. 固定大小线程池
ExecutorService fixedPool = Executors.newFixedThreadPool(4);

// 2. 缓存线程池（不推荐，可能创建过多线程）
ExecutorService cachedPool = Executors.newCachedThreadPool();

// 3. 单线程池
ExecutorService singlePool = Executors.newSingleThreadExecutor();

// 4. 定时任务线程池
ScheduledExecutorService scheduledPool =
    Executors.newScheduledThreadPool(2);
scheduledPool.scheduleAtFixedRate(() -> {
    System.out.println("Scheduled task");
}, 0, 1, TimeUnit.SECONDS);

```


9.3 线程池监控

```
public class ThreadPoolMonitor {
    public static void monitor(ThreadPoolExecutor executor) {
        System.out.println("Pool Size: " +
            executor.getPoolSize());
        System.out.println("Active Threads: " +
            executor.getActiveCount());
        System.out.println("Completed Tasks: " +
            executor.getCompletedTaskCount());
        System.out.println("Queue Size: " +
            executor.getQueue().size());
    }
}
```

十、CompletableFuture (异步编程)

10.1 基本用法

```
import java.util.concurrent.CompletableFuture;

public class CompletableFutureDemo {
    public static void main(String[] args) throws Exception {
        // 异步执行任务
        CompletableFuture<String> future = CompletableFuture
            .supplyAsync(() -> {
                try { Thread.sleep(1000); } catch
                (InterruptedException e) { }
                return "Hello Async";
            })
            .thenApply(result -> result + " world")
            .thenApply(String::toUpperCase);

        System.out.println("Result: " + future.get()); // 阻塞等待

        // 组合多个异步任务
        CompletableFuture<String> future1 =
            CompletableFuture.supplyAsync(() -> "Task1");
        CompletableFuture<String> future2 =
            CompletableFuture.supplyAsync(() -> "Task2");
    }
}
```

```

        CompletableFuture<String> combined =
future1.thenCombine(future2, (s1, s2) -> s1 + " + " + s2);
        System.out.println("Combined: " + combined.get());
    }
}

```

10.2 异常处理

```

CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> {
        throw new RuntimeException("Error occurred");
    })
    .exceptionally(throwable -> {
        System.out.println("Caught exception: " +
throwable.getMessage());
        return "Default value";
    });

```

十一、并发设计模式

11.1 单例模式（线程安全）

```

public class Singleton {
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) { // 第一次检查
            synchronized (Singleton.class) {
                if (instance == null) { // 第二次检查
                    instance = new Singleton(); // volatile 防止重
排序
                }
            }
        }
        return instance;
    }
}

// 或使用枚举（推荐）
public enum SingletonEnum {
    INSTANCE;
}

```

```
public void doSomething() { }  
}
```

11.2 生产者-消费者模式

```
public class ProducerConsumerPattern {  
    private final Queue<String> queue = new ArrayBlockingQueue<>  
(10);  
  
    class Producer implements Runnable {  
        @Override  
        public void run() {  
            try {  
                for (int i = 0; i < 100; i++) {  
                    queue.put("Item-" + i);  
                    System.out.println("Produced: Item-" + i);  
                    Thread.sleep(100);  
                }  
            } catch (InterruptedException e) {  
                Thread.currentThread().interrupt();  
            }  
        }  
    }  
  
    class Consumer implements Runnable {  
        @Override  
        public void run() {  
            try {  
                while (true) {  
                    String item = queue.take();  
                    System.out.println("Consumed: " + item);  
                    Thread.sleep(200);  
                }  
            } catch (InterruptedException e) {  
                Thread.currentThread().interrupt();  
            }  
        }  
    }  
}
```

十二、性能调优与最佳实践

12.1 性能监控

```
// 使用 JMH 进行微基准测试
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
public class PerformanceTest {
    @Benchmark
    public void testSynchronized() {
        synchronized (this) {
            // 测试代码
        }
    }

    @Benchmark
    public void testReentrantLock() {
        lock.lock();
        try {
            // 测试代码
        } finally {
            lock.unlock();
        }
    }
}
```

12.2 最佳实践

场景	推荐方案
高并发读多写少	ConcurrentHashMap + ReadwriteLock
计数器	LongAdder > AtomicLong > synchronized
线程池	自定义配置，避免 Executors 默认池
异步处理	CompletableFuture + 自定义线程池
锁粒度	分段锁、读写分离

十三、总结：多线程核心要点

概念	关键点
三大问题	可见性 (volatile) 、原子性 (synchronized/Atomic) 、有序性 (happens-before)
同步机制	synchronized (简单) 、ReentrantLock (灵活) 、JUC 工具类
并发集合	ConcurrentHashMap、BlockingQueue、CopyOnWriteArrayList
线程池	合理配置核心/最大线程数、队列大小、拒绝策略
异步编程	CompletableFuture、响应式编程 (Reactor)

 一句话理解多线程：
“并发是双刃剑，正确使用提升性能，错误使用引发灾难。”

附录：学习资源

- **书籍**：《Java 并发编程实战》、《Java 并发编程的艺术》
- **工具**：JMH (性能测试) 、Arthas (线上诊断)
- **框架**：Disruptor (高性能队列) 、LMAX (异步处理)

如需进一步了解：

- 如何实现一个自定义的阻塞队列？
- CompletableFuture 的线程池配置？
- JVM 并发调优实战？

一、线程与进程

1. 基本概念

- **进程 (Process)** ：操作系统分配资源的最小单位。
- **线程 (Thread)** ：CPU 调度的最小单位，一个进程可包含多个线程。
- **多线程 (Multithreading)** ：在同一进程中并发执行多个线程，提高 CPU 利用率。

2. 并发与并行

- **并发 (Concurrency)**：在同一时间段内交替执行多个任务。
- **并行 (Parallelism)**：在多核 CPU 上真正同时执行多个任务。

二、创建线程的方式

方式	实现方式	优点	缺点
继承 <code>Thread</code>	重写 <code>run()</code> 方法	简单直观	不支持多继承
实现 <code>Runnable</code>	将任务与线程分离	灵活	不能直接返回结果
实现 <code>Callable</code>	<code>FutureTask</code> 支持返回值与异常	功能更强	稍复杂
线程池 (<code>ExecutorService</code>)	使用线程池管理线程	高效复用	配置较复杂

三、线程的生命周期

状态	说明
NEW	新建状态
RUNNABLE	就绪状态，等待 CPU 调度
BLOCKED	被锁阻塞（等待进入 synchronized）
WAITING	无限期等待（如 <code>wait()</code> ）
TIMED_WAITING	限时等待（如 <code>sleep()</code> 、 <code>join(timeout)</code> ）
TERMINATED	线程结束

四、线程同步与锁机制

1. synchronized

- 用于方法或代码块，保证同一时刻只有一个线程访问。
- 锁对象：实例锁或类锁（Class 对象）。

2. ReentrantLock

- 可重入锁，提供更灵活的加锁与解锁控制。
- 支持可中断锁、尝试锁、公平锁。

3. volatile

- 保证变量的可见性与禁止指令重排序。
- 不保证原子性。

4. 原子类（Atomic）

- 位于 `java.util.concurrent.atomic`，如：
 - `AtomicInteger`
 - `AtomicLong`
 - `AtomicReference`
- 通过 CAS（Compare-And-Swap）实现无锁并发。

五、线程间通信

方式	描述
<code>wait()</code> / <code>notify()</code> / <code>notifyAll()</code>	依赖对象监视器
<code>Condition.await()</code> / <code>signal()</code>	与 <code>Lock</code> 配合使用
阻塞队列（ <code>BlockingQueue</code> ）	常用于生产者-消费者模型
<code>CountDownLatch</code> / <code>CyclicBarrier</code>	线程同步与等待工具类

六、线程池详解

线程池位于 `java.util.concurrent` 包中，由 `ThreadPoolExecutor` 实现。

1. 核心参数

- `corePoolSize`: 核心线程数
- `maximumPoolSize`: 最大线程数
- `keepAliveTime`: 空闲线程存活时间
- `workQueue`: 任务队列
- `handler`: 拒绝策略

2. 拒绝策略

- `AbortPolicy`: 直接抛异常
- `CallerRunsPolicy`: 由调用线程执行
- `DiscardPolicy`: 丢弃任务
- `DiscardOldestPolicy`: 丢弃队列中最老的任务

3. 常用线程池类型

- `newFixedThreadPool`: 固定大小线程池
- `newCachedThreadPool`: 缓存线程池（自动扩容）
- `newScheduledThreadPool`: 定时任务线程池
- `newSingleThreadExecutor`: 单线程顺序执行

七、JUC 高级并发工具

类	说明
<code>Semaphore</code>	信号量，控制并发访问数
<code>CountDownLatch</code>	倒计时锁，等待多个线程完成
<code>CyclicBarrier</code>	栅栏，等待一组线程到达屏障点
<code>Exchanger</code>	两个线程间交换数据
<code>CompletableFuture</code>	异步编程框架

八、线程安全与问题

问题类型	原因	解决方案
可见性问题	缓存未同步	使用 <code>volatile</code> 或同步块
原子性问题	多步操作非原子	使用锁或原子类
有序性问题	指令重排	<code>volatile</code> 、内存屏障
死锁	多线程互相持锁	避免循环等待或使用 <code>tryLock()</code>

九、线程调优与监控

1. JVM工具

- `jconsole`、`jvisualvm`、`jstack`

2. 指标

- 活跃线程数
- 阻塞/等待线程数
- 线程池队列大小

3. 调优手段

- 控制线程数量 (CPU核心数×2)
- 合理设置线程池参数
- 减少锁竞争

十、面试高频题

1. `synchronized` 与 `ReentrantLock` 区别?
2. `volatile` 原理? 能保证原子性吗?
3. 什么是 CAS? ABA 问题如何解决?
4. 线程池工作原理?
5. 如何避免死锁?
6. `ThreadLocal` 是什么? 有什么风险?
7. `CompletableFuture` 和 `Future` 的区别?

Java 多线程详解（从底层原理到生产实战）

Java 多线程是并发编程的核心，其本质是通过充分利用 CPU 多核资源，让多个任务“同时”执行，从而提升程序吞吐量（如高并发接口）和响应速度（如异步任务处理）。本文从底层原理、核心 API、线程安全、并发工具到生产实践，全面拆解 Java 多线程知识体系，兼顾深度与实用性。

一、多线程底层核心原理

要学好多线程，先搞懂底层逻辑：线程的本质、操作系统调度机制、Java 线程与操作系统线程的关联。

1. 线程与进程的本质区别

维度	进程 (Process)	线程 (Thread)
资源分配	操作系统资源分配的最小单位（独立内存空间、文件句柄）	CPU 调度的最小单位（共享进程资源）
开销	创建 / 销毁开销大（需分配资源）	创建 / 销毁开销小（复用进程资源）
通信方式	复杂（管道、Socket、共享内存）	简单（直接访问进程堆、方法区共享资源）
独立性	高（一个进程崩溃不影响其他进程）	低（线程崩溃可能导致整个进程崩溃）

核心结论：Java 程序的运行载体是进程（JVM 进程），多线程是 JVM 进程内的多个执行流，共享堆内存（对象）、方法区（类元数据），但每个线程有独立的程序计数器（PC）、虚拟机栈（局部变量）、本地方法栈。

2. 线程调度机制（CPU 如何分配时间片）

操作系统通过“时间片轮转”或“优先级调度”分配 CPU 资源给线程，核心概念：

- 时间片：**CPU 分配给线程的执行时间（通常 10-100ms），时间片结束后线程切换；

- **上下文切换**：线程切换时，CPU 保存当前线程状态（PC、栈指针等），恢复目标线程状态的过程（有性能开销，线程数过多会导致开销激增）；
- **就绪队列**：线程等待 CPU 调度时，进入就绪队列（`RUNNABLE` 状态的核心）；
- **阻塞队列**：线程因等待资源（锁、IO）进入阻塞队列（`BLOCKED / WAITING` 状态），资源就绪后回到就绪队列。

3. Java 线程与操作系统线程的映射关系

Java 线程的实现依赖操作系统内核线程（1:1 映射模型，JDK 1.2 后统一）：

- 调用 `Thread.start()` 时，JVM 会调用操作系统 API 创建内核线程（`pthread_create` 等）；
- Java 线程的状态（`RUNNABLE / BLOCKED` 等）本质是对内核线程状态的封装；
- 线程的调度（CPU 分配）由操作系统完成，JVM 仅负责线程的创建、销毁和状态管理。

二、Java 线程核心 API 与生命周期

1. Thread 类核心 API（常用方法）

方法名	作用	注意事项
<code>start()</code>	启动线程 (JVM 调用 <code>run()</code> 方法)	不能重复调用（否则抛 <code>IllegalThreadStateException</code> ）
<code>run()</code>	线程执行的任务逻辑	直接调用仅为普通方法，不会启动新线程
<code>sleep(long ms)</code>	线程休眠指定时间 (释放 CPU，不释放锁)	会抛出 <code>InterruptedException</code> ，需捕获或抛出
<code>join() / join(long ms)</code>	等待目标线程执行完毕（限时 / 无限时）	调用线程进入 <code>WAITING / TIMED_WAITING</code>

方法名	作用	注意事项
<code>yield()</code>	主动放弃当前时间片，回到就绪队列	仅为建议，CPU 可能忽略该请求
<code>setDaemon(boolean)</code>	设置为守护线程（默认用户线程）	必须在 <code>start()</code> 前设置，主线程退出后守护线程自动终止
<code>setPriority(int)</code>	设置线程优先级（1-10，默认 5）	依赖操作系统，优先级高不一定先执行
<code>interrupt()</code>	中断线程（设置中断标志位）	不会强制终止线程，需配合 <code>isInterrupted()</code> 判断
<code>isInterrupted()</code>	判断线程是否被中断（不清除标志位）	-
<code>interrupted()</code>	判断当前线程是否被中断（清除标志位）	静态方法，调用后标志位重置为 false

关键 API 示例 (interrupt () 中断线程)

java

运行

```

public class InterruptDemo {
    public static void main(String[] args) throws
InterruptedException {
        Thread thread = new Thread(() -> {
            // 循环中检查中断标志位
            while (!Thread.currentThread().isInterrupted()) {
                System.out.println("线程执行中...");
                try {
                    Thread.sleep(500); // sleep 期间被中断会抛出异常，
清除标志位
                } catch (InterruptedException e) {
                    System.out.println("线程被中断，退出循环");
                    // 可选：重新设置中断标志位（因 sleep 会清除标志位）
                    Thread.currentThread().interrupt();
                    break;
                }
            }
        });

        thread.start();
        Thread.sleep(2000); // 主线程休眠 2 秒
        thread.interrupt(); // 中断线程
    }
}

```

2. 线程生命周期（6 种状态 + 完整转换逻辑）

Java 线程状态定义在 `Thread.State` 枚举中，是多线程编程的核心，需牢记“状态转换的触发条件”：

（1）6 种核心状态

状态名称	核心说明
NEW	线程对象已创建，但未调用 <code>start()</code> （仅存在于 JVM 层面，未映射到内核线程）
RUNNABLE	调用 <code>start()</code> 后进入，包含“就绪”（等待 CPU 调度）和“运行中”（占用 CPU）
BLOCKED	仅因竞争 <code>synchronized</code> 锁失败而阻塞（等待锁释放）

状态名称	核心说明
WAITING	无限期等待被唤醒（如 <code>wait()</code> 无参、 <code>join()</code> 无参），释放 CPU 和锁
TIMED_WAITING	限时等待（如 <code>sleep(ms)</code> 、 <code>wait(ms)</code> ），超时自动唤醒或被主动唤醒
TERMINATED	线程执行完毕（ <code>run()</code> 结束）或异常终止（无捕获异常）

(2) 完整状态转换表（无遗漏）

当前状态	目标状态	触发条件
NEW	RUNNABLE	调用 <code>thread.start()</code>
RUNNABLE	BLOCKED	竞争 <code>synchronized</code> 锁失败（如线程 A 持有锁，线程 B 抢不到）
RUNNABLE	WAITING	1. 调用 <code>Object.wait()</code> （无参，需在 <code>synchronized</code> 块内）；2. 调用 <code>Thread.join()</code> （无参）；3. 调用 <code>LockSupport.park()</code>
RUNNABLE	TIMED_WAITING	1. <code>Thread.sleep(long ms)</code> ；2. <code>Object.wait(long ms)</code> ；3. <code>Thread.join(long ms)</code> ；4. <code>LockSupport.parkNanos()</code> ；5. <code>LockSupport.parkUntil()</code>
RUNNABLE	TERMINATED	1. <code>run()</code> 方法正常执行完毕；2. 抛出未捕获异常（如 NPE、OOM）
BLOCKED	RUNNABLE	持有 <code>synchronized</code> 锁的线程释放锁，当前线程抢到锁
WAITING	RUNNABLE	1. 其他线程调用 <code>Object.notify()/notifyAll()</code> （对应 <code>wait()</code> ）；2. 目标线程执行完毕（对应 <code>join()</code> ）；3. 其他线程调用 <code>LockSupport.unpark(thread)</code>
TIMED_WAITING	RUNNABLE	1. 等待超时；2. 其他线程唤醒（ <code>notify()/unpark()</code> 等）

当前状态	目标状态	触发条件
TERMINATED	无	线程终止后不可逆转，再次调用 <code>start()</code> 会抛异常

(3) 易错点澄清

- `BLOCKED` 仅针对 `synchronized`：等待 `ReentrantLock` 锁时，线程状态是 `WAITING/TIMED_WAITING`（不是 `BLOCKED`）；
- `sleep()` 不释放锁：`Thread.sleep(ms)` 仅释放 CPU，持有锁时不会释放（如 `synchronized` 块内 `sleep`，其他线程仍抢不到锁）；
- `wait()` 必须在 `synchronized` 块内：否则抛 `IllegalMonitorStateException`，且调用后释放锁；
- 终止线程的正确方式：通过 `interrupt()` 设标志位（配合业务判断），而非 `stop()`（已废弃，强制终止会导致资源泄漏）。

三、线程创建的 4 种方式（含生产环境最佳实践）

Java 提供 4 种创建线程的方式，核心是“定义任务”和“启动线程”，生产环境优先使用线程池。

1. 方式 1：继承 Thread 类（基础方式）

- 核心逻辑：继承 `Thread` 类，重写 `run()` 方法定义任务，调用 `start()` 启动线程；
- 缺点：Java 单继承限制，任务与线程耦合（不便于复用）；
- 示例：

java

运行

```
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println(Thread.currentThread().getName() + " 执行任务");
    }

    public static void main(String[] args) {
        new MyThread().start(); // 输出: Thread-0 执行任务
    }
}
```

2. 方式 2：实现 Runnable 接口（解耦方式）

- 核心逻辑：Runnable 是函数式接口（仅 run() 方法），定义任务后传入 Thread 启动；
- 优点：无单继承限制，任务与线程解耦（可复用任务）；
- 示例（Lambda 简化, JDK 8+）：

java

运行

```
// Lambda 表达式定义 Runnable 任务（无需实现类）
Runnable task = () ->
    System.out.println(Thread.currentThread().getName() + " 执行任务");
new Thread(task, "自定义线程名").start(); // 输出: 自定义线程名 执行任务
```

3. 方式 3：实现 Callable 接口（有返回值 / 可抛异常）

- 核心逻辑：Callable<V> 支持返回值（V 为类型）和抛出受检异常，需配合 FutureTask 包装（兼具 Runnable 和 Future 特性）；
- 适用场景：异步计算（如并行计算求和、异步查询数据库）；
- 示例：

java

运行

```
import java.util.concurrent.Callable;
import java.util.concurrent.FutureTask;

// 实现 Callable, 返回 Integer 类型结果
class SumCallable implements Callable<Integer> {
    private int start, end;
    public SumCallable(int start, int end) {
        this.start = start;
        this.end = end;
    }

    @Override
    public Integer call() throws Exception {
        int sum = 0;
        for (int i = start; i <= end; i++) sum += i;
        return sum;
    }

    public static void main(String[] args) throws Exception {
        // 包装 Callable 为 FutureTask
        FutureTask<Integer> futureTask = new FutureTask<>(new
SumCallable(1, 100));
        new Thread(futureTask).start(); // 启动线程

        // get() 阻塞获取结果（直到任务完成）
        System.out.println("1-100 求和结果: " + futureTask.get());
        // 输出 5050
    }
}
```

4. 方式 4：线程池（ExecutorService，生产环境首选）

直接创建线程（方式 1-3）的缺陷：线程创建 / 销毁开销大、无界线程可能导致 OOM、无法统一管理。**线程池**通过复用线程、控制并发数、缓冲任务，解决上述问题。

（1）线程池核心组件（ThreadPoolExecutor）

java

运行

```
// 自定义线程池（生产环境推荐，而非 Executors 工具类）
ThreadPoolExecutor executor = new ThreadPoolExecutor(
    2,                // 核心线程数（常驻线程，即使空闲也不销毁）
    5,                // 最大线程数（核心线程+临时线程的上限）
    60,               // 临时线程空闲存活时间
    TimeUnit.SECONDS, // 存活时间单位
    new ArrayBlockingQueue<>(10), // 任务队列（有界，避免内存溢出）
    Executors.defaultThreadFactory(), // 线程工厂（创建线程）
    new ThreadPoolExecutor.AbortPolicy() // 拒绝策略（任务满时抛异常）
);
```

（2）拒绝策略（4 种默认策略）

策略名称	作用	适用场景
AbortPolicy	抛出 RejectedExecutionException	核心业务（需感知任务失败）
CallerRunsPolicy	由提交任务的线程（如主线程）执行	非核心业务（允许任务降级）

策略名称	作用	适用场景
<code>DiscardPolicy</code>	直接丢弃新任务（无提示）	不重要任务 （如日志收集）
<code>DiscardOldestPolicy</code>	丢弃队列中最旧的任务，加入新任务	任务时效性强 （如实时数据处理）

(3) 线程池使用示例（提交任务）

java

运行

```
// 提交 Runnable 任务（无返回值）
executor.submit(() -> System.out.println("执行无返回值任务"));

// 提交 Callable 任务（有返回值）
Future<Integer> future = executor.submit(() -> 100 + 200);
System.out.println("任务结果: " + future.get()); // 输出 300

// 关闭线程池（必须调用，否则 JVM 不会退出）
executor.shutdown(); // 执行完已提交任务，不再接受新任务
// executor.shutdownNow(); // 立即关闭，中断正在执行的任务（慎用）
```

(4) Executors 工具类（入门用，生产慎用）

`Executors` 提供快速创建线程池的方法，但存在潜在风险（如 `newCachedThreadPool` 无界线程可能导致 OOM）：

- `Executors.newFixedThreadPool(n)`：固定大小线程池；
- `Executors.newSingleThreadExecutor()`：单线程池；
- `Executors.newCachedThreadPool()`：缓存线程池（核心线程 0，临时线程无界）；

- `Executors.newScheduledThreadPool(n)`：定时任务线程池。

四、线程安全与同步机制（核心重点）

多线程并发访问共享资源时，会因“竞态条件”（Race Condition）导致线程不安全（如脏读、重复计算）。Java 提供多种同步机制，核心是保证“原子性、可见性、有序性”（线程安全三要素）。

1. 线程安全三要素（必须理解）

- **原子性**：操作不可分割（如 `i++` 不是原子操作，拆分为“读 - 改 - 写”三步，可能被线程打断）；
- **可见性**：一个线程修改共享资源后，其他线程能立即看到最新值（避免 CPU 缓存导致的旧值读取）；
- **有序性**：线程执行顺序符合代码逻辑（避免编译器 / CPU 重排序导致的执行混乱）。

2. 同步机制 1：synchronized（隐式锁，基础首选）

`synchronized` 是 Java 内置隐式锁（监视器锁），自动实现锁的获取和释放，支持修饰方法、代码块，能保证原子性、可见性、有序性。

（1）三种使用方式

java

运行

```
class SynchronizedDemo {
    private int count = 0;
    private final Object lock = new Object(); // 专用锁对象（推荐）

    // 1. 修饰实例方法（锁对象：this）
    public synchronized void increment1() {
        count++;
    }
}
```

```
// 2. 修饰静态方法（锁对象：SynchronizedDemo.class）
public static synchronized void increment2() {
    // 静态变量操作
}

// 3. 修饰代码块（锁对象：自定义对象，锁粒度最细，推荐）
public void increment3() {
    synchronized (lock) { // 锁定 lock 对象，仅代码块内同步
        count++;
    }
}
}
```

(2) synchronized 核心原理

- 锁的本质：对象头中的“监视器锁（monitor）”，线程获取锁时进入 monitor 的 EntryList 阻塞，释放锁时唤醒阻塞线程；
- 自动释放锁：无论正常执行还是抛出异常，JVM 都会自动释放锁（避免死锁风险）；
- 可见性保障：释放锁时将工作内存的修改同步到主内存，获取锁时清空工作内存，从主内存加载最新值；
- 有序性保障：禁止重排序（通过内存屏障实现）；
- 可重入性：同一线程可多次获取同一把锁（如同步方法调用同步方法，不会死锁）。

3. 同步机制 2：ReentrantLock（显式锁，功能更强）

`ReentrantLock` 是 `java.util.concurrent.locks` 包下的显式锁，需手动调用 `lock()` 获取锁、`unlock()` 释放锁（必须在 `finally` 中释放），功能比 `synchronized` 更灵活。

(1) 核心特性

- 可重入：与 `synchronized` 一致；
- 公平锁 / 非公平锁：默认非公平锁（效率高），构造函数 `new ReentrantLock(true)` 可设为公平锁（按线程等待顺序获取锁）；
- 超时锁：`tryLock(long ms, TimeUnit)` 超时未获取锁则返回 `false`（避免无限阻塞）；
- 条件变量：通过 `newCondition()` 获取 `Condition` 对象，支持精准唤醒线程（而非 `notify()` 随机唤醒）；

- 锁状态查询：支持 `isLocked()`、`getHoldCount()` 等方法查询锁状态。

(2) 示例代码（条件变量实现生产者 - 消费者）

java

运行

```
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

class BlockingQueue<T> {
    private final T[] queue;
    private int count = 0;
    private int putIdx = 0;
    private int takeIdx = 0;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition(); // 队列
    // 未满条件
    private final Condition notEmpty = lock.newCondition(); // 队
    // 列非空条件

    @SuppressWarnings("unchecked")
    public BlockingQueue(int capacity) {
        queue = (T[]) new Object[capacity];
    }

    // 生产者放入元素
    public void put(T t) throws InterruptedException {
        lock.lock();
        try {
            while (count == queue.length) {
                notFull.await(); // 队列满，等待未满条件
            }
            queue[putIdx] = t;
            putIdx = (putIdx + 1) % queue.length;
            count++;
        }
    }
}
```

```

        notEmpty.signal(); // 唤醒消费者（队列非空）
    } finally {
        lock.unlock(); // 必须在 finally 中释放锁
    }
}

// 消费者取出元素
public T take() throws InterruptedException {
    lock.lock();
    try {
        while (count == 0) {
            notEmpty.await(); // 队列空，等待非空条件
        }
        T t = queue[takeIdx];
        takeIdx = (takeIdx + 1) % queue.length;
        count--;
        notFull.signal(); // 唤醒生产者（队列未满）
        return t;
    } finally {
        lock.unlock();
    }
}
}

```

4. 同步机制 3: volatile（轻量级同步）

`volatile` 修饰变量，**不保证原子性**，仅保证可见性和有序性，适用于“单线程写、多线程读”场景（如状态标记位）。

(1) 核心作用

- 可见性：修改 `volatile` 变量后，立即同步到主内存；其他线程读取时，从主内存加载最新值（避免 CPU 缓存旧值）；
- 有序性：禁止 `volatile` 变量前后的指令重排序（通过内存屏障实现）；
- 不保证原子性：如 `volatile int i = 0; i++` 仍存在线程安全问题（需结合原子类或锁）。

(2) 典型使用场景

java

运行

// 场景 1: 状态标记位（线程停止信号）

```
class volatileFlagDemo {  
    private volatile boolean isStop = false; // volatile 保证可见性  
  
    public void run() {  
        while (!isStop) {  
            System.out.println("线程执行中...");  
        }  
        System.out.println("线程停止");  
    }  
  
    public void stop() {  
        isStop = true; // 修改后，线程立即看到最新值  
    }  
}
```

// 场景 2: 单例模式双重检查锁定（DCL）

```
class Singleton {  
    private static volatile Singleton instance; // 禁止重排序  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null) { // 第一次检查（无锁，提高效率）  
            synchronized (Singleton.class) { // 加锁  
                if (instance == null) { // 第二次检查（避免多线程创建多个实例）  
                    instance = new Singleton(); // volatile 禁止重排序（避免半初始化对象）  
                }  
            }  
        }  
        return instance;  
    }  
}
```


5. 同步机制 4：原子类（AtomicXXX，无锁同步）

`java.util.concurrent.atomic` 包下的原子类，基于 CAS（Compare And Swap）操作实现无锁同步，保证原子性，性能优于锁（避免上下文切换）。

（1）CAS 原理（乐观锁）

CAS 是“比较并替换”的原子操作，底层通过 CPU 指令 `cmpxchg` 实现：

1. 假设变量当前值为 `expected`，期望修改为 `newValue`；
2. 若变量实际值 == `expected`，则修改为 `newValue`，返回 `true`；
3. 若不一致，不修改，返回 `false`（可重试或放弃）。

（2）常用原子类

- 基本类型：`AtomicInteger`、`AtomicLong`、`AtomicBoolean`；
- 引用类型：`AtomicReference`（原子修改对象引用）、`AtomicStampedReference`（解决 CAS ABA 问题）；
- 数组类型：`AtomicIntegerArray`、`AtomicLongArray`；
- 字段更新器：`AtomicIntegerFieldUpdater`（原子修改对象的非原子字段）；
- 高性能计数：`LongAdder`（高并发场景下比 `AtomicLong` 性能更优，分段计数）。

（3）示例代码（AtomicInteger 原子自增）

java

运行

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicDemo {
```

```

        private static final AtomicInteger count = new
        AtomicInteger(0);

        public static void main(String[] args) throws
        InterruptedException {
            ExecutorService executor =
            Executors.newFixedThreadPool(10);

            // 10 个线程各执行 1000 次自增
            for (int i = 0; i < 10; i++) {
                executor.submit(() -> {
                    for (int j = 0; j < 1000; j++) {
                        count.incrementAndGet(); // 原子自增 (count++)
                    }
                });
            }

            executor.shutdown();
            executor.awaitTermination(1, TimeUnit.SECONDS);

            System.out.println("最终计数: " + count.get()); // 输出
            10000 (线程安全)
        }
    }
}

```

五、Java 并发工具类（高效协作）

JDK `java.util.concurrent` 包提供多种并发工具类，避免重复造轮子，核心用于线程协作、限流、并发集合等场景。

1. CountdownLatch（倒计时器）

- 作用：等待多个线程完成任务后，主线程再继续执行（如主线程等待 10 个子线程数据处理完成后汇总）；
- 核心方法：`countDown()`（线程完成任务后调用，倒计时减 1）、`await()`（主线程阻塞直到倒计时为 0）；
- 示例：

java

运行

```

import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class CountDownLatchDemo {
    public static void main(String[] args) throws
InterruptedException {
        int threadNum = 3;
        CountDownLatch latch = new CountDownLatch(threadNum);

        ExecutorService executor =
Executors.newFixedThreadPool(threadNum);
        for (int i = 0; i < threadNum; i++) {
            int taskId = i;
            executor.submit(() -> {
                try {
                    System.out.println("任务" + taskId + "执行中");
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                } finally {
                    latch.countDown(); // 倒计时减 1 (必须在 finally
中)
                }
            });
        }

        System.out.println("主线程等待所有任务完成...");
        latch.await(); // 阻塞直到所有任务完成
        System.out.println("所有任务执行完毕，主线程继续");
        executor.shutdown();
    }
}

```

2. CyclicBarrier (循环栅栏)

- 作用：让多个线程在“栅栏”处等待，所有线程到达后一起继续执行（可循环使用，如多线程分阶段执行任务）；

- 核心方法: `await()` (线程到达栅栏后阻塞)、构造函数 `CyclicBarrier(int parties, Runnable action)` (所有线程到达后执行 `action`) ;
- 区别 `CountDownLatch`: `CyclicBarrier` 可循环使用, `CountDownLatch` 倒计时完成后不可复用。

3. Semaphore (信号量)

- 作用: 控制同时访问某个资源的线程数 (限流场景, 如限制最大并发连接数) ;
- 核心方法: `acquire()` (获取许可, 无许可则阻塞)、`release()` (释放许可)、`tryAcquire()` (尝试获取许可, 无则返回 `false`) ;
- 示例 (限流 2 个并发) :

java

运行

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Semaphore;

public class SemaphoreDemo {
    public static void main(String[] args) {
        int maxConcurrent = 2; // 最大并发数 (许可数)
        Semaphore semaphore = new Semaphore(maxConcurrent);

        ExecutorService executor =
            Executors.newFixedThreadPool(5);
        for (int i = 0; i < 5; i++) {
            int taskId = i;
            executor.submit(() -> {
                try {
                    semaphore.acquire(); // 获取许可
                    System.out.println("任务" + taskId + "执行中 (并发数: " + (2 - semaphore.availablePermits()) + ")");
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                semaphore.release();
            });
        }
    }
}
```

```
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            semaphore.release(); // 释放许可
        }
    });

    executor.shutdown();
}
```

4. 并发集合（线程安全的集合）

集合类	线程安全实现方式	适用场景
<code>ConcurrentHashMap</code>	JDK 8+: CAS + synchronized (桶级锁)	高并发读写 (如缓存)
<code>CopyOnWriteArrayList</code>	写操作复制全表，读操作无锁	读多写少 (如 配置列表)
<code>CopyOnWriteArraySet</code>	基于 <code>CopyOnWriteArrayList</code>	读多写少的 Set 场景
<code>LinkedBlockingQueue</code>	基于 <code>ReentrantLock</code> + <code>Condition</code>	生产者 - 消费 者模型 (有界 队列)
<code>ArrayBlockingQueue</code>	基于 <code>ReentrantLock</code> + <code>Condition</code>	固定容量的生 产者 - 消费者 模型
<code>ConcurrentLinkedQueue</code>	无锁 (CAS) 实现	高并发无界队 列 (如任务队 列)

示例（ConcurrentHashMap 高并发读写）

java

运行

```
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ConcurrentMapDemo {
    public static void main(String[] args) throws
InterruptedException {
        Map<String, Integer> map = new ConcurrentHashMap<>();
        ExecutorService executor =
Executors.newFixedThreadPool(10);

        // 10 个线程同时 put 数据
        for (int i = 0; i < 10; i++) {
            int num = i;
            executor.submit(() -> map.put("key" + num, num));
        }

        executor.shutdown();
        executor.awaitTermination(1, TimeUnit.SECONDS);

        System.out.println("map 大小: " + map.size()); // 输出 10
        (线程安全)
    }
}
```

六、多线程常见问题与解决方案（生产避坑）

1. 死锁（Deadlock）

现象：

多个线程互相持有对方需要的锁，无限期阻塞（如线程 A 持有锁 1，等待锁 2；线程 B 持有锁 2，等待锁 1）。

产生条件（缺一不可）：

1. 互斥条件：资源只能被一个线程持有；
2. 请求与保持条件：线程持有一个资源，同时请求另一个；
3. 不可剥夺条件：资源不能被强制剥夺；
4. 循环等待条件：线程间形成循环等待链。

解决方案：

- 破坏循环等待：统一锁的获取顺序（如所有线程先获取锁 1，再获取锁 2）；
- 破坏请求与保持：一次性获取所有需要的锁；
- 超时锁：使用 `ReentrantLock.tryLock(ms)`，超时未获取锁则释放已持有锁；
- 监控死锁：`jstack <pid>` 命令查看线程状态，定位死锁线程和锁资源。

2. 线程泄漏（Thread Leak）

现象：

线程创建后未正常终止，长期占用资源（如线程池核心线程持有任务引用，任务未结束导致线程无法回收）。

常见原因：

- 任务中存在无限循环（无退出条件）；
- 线程阻塞在 `wait()` / `join()` 等方法，未被唤醒；
- 自定义线程未设为守护线程，主线程退出后仍运行。

解决方案：

- 任务中添加退出条件（如 `volatile` 标记位）；
- 使用超时机制（`wait(ms)`、`tryLock(ms)`）避免无限阻塞；
- 合理设置线程池参数（如临时线程存活时间）；
- 非核心线程设为守护线程（`thread.setDaemon(true)`）。

3. 上下文切换过多

现象：

线程数远超 CPU 核心数，CPU 频繁切换线程，上下文切换开销占比高，吞吐量下降。

解决方案：

- 控制线程数：CPU 密集型任务（如计算）线程数 = CPU 核心数；IO 密集型任务（如数据库查询）线程数 = CPU 核心数 * 2；
- 减少锁竞争：使用无锁并发（原子类、`ConcurrentHashMap`）、缩小锁粒度；
- 批量处理任务：减少任务提交次数，降低线程切换频率；
- 异步化：IO 操作异步化（如 `CompletableFuture`），减少线程阻塞。

4. 线程安全问题排查工具

- `jstack`：查看线程状态（`BLOCKED`/`WAITING`）、定位死锁；
- `Arthas`：实时查看线程状态、锁竞争（`thread -b` 查看阻塞线程）；
- `jvisualvm`：可视化监控线程、GC、内存使用；
- `MAT`：分析堆快照，排查因线程泄漏导致的内存溢出。

七、生产环境多线程最佳实践

1. **优先使用线程池，而非直接创建线程**：线程池复用线程、控制并发数、统一管理，避免 OOM；
2. **线程池必须自定义参数**：拒绝使用 `Executors`，直接使用 `ThreadPoolExecutor`，指定有界队列和合理拒绝策略；
3. 减少锁竞争
：
 - 锁粒度最小化（`synchronized` 修饰代码块而非方法）；
 - 读多写少用 `CopyOnWriteArrayList`/`ConcurrentHashMap`；
 - 高并发计数用 `LongAdder` 替代 `AtomicLong`；
4. 避免线程阻塞
：
 - IO 操作（数据库、网络）异步化（`CompletableFuture`、响应式编程）；
 - 避免无限期阻塞，使用超时机制；

5. 线程安全优先用简单方案

:

- 无共享资源 → 天然线程安全;
- 共享资源只读 → 天然线程安全;
- 共享资源需修改 → 优先用原子类, 其次 `synchronized`, 复杂场景用 `ReentrantLock`;

6. 避免使用 `ThreadLocal` 不当导致内存泄漏

:

- 线程池场景下, `ThreadLocal` 需在任务结束后调用 `remove()`;
- 原因: 线程池核心线程复用, `ThreadLocal` 绑定的变量会长期存在, 导致内存泄漏。

八、总结

Java 多线程的核心是“平衡并发与线程安全”, 关键点可归纳为:

1. **底层原理**: Java 线程是内核线程的 1:1 映射, 线程调度由操作系统完成, 上下文切换是性能开销的核心;
2. **线程创建**: 生产环境首选线程池 (`ThreadPoolExecutor`), 避免直接创建线程;
3. **线程安全**: 通过 `synchronized/ReentrantLock` 保证原子性, `volatile` 保证可见性 / 有序性, 原子类实现无锁同步;
4. **并发工具**: `CountDownLatch/CyclicBarrier` 用于线程协作, `Semaphore` 用于限流, 并发集合用于线程安全存储;
5. **问题排查**: 死锁用 `jstack`, 线程泄漏用 `jmap+MAT`, 性能问题用 `Arthas`;
6. **最佳实践**: 控制线程数、减少锁竞争、避免阻塞、优先简单方案。

111

1 线程与进程：一句话区分

- **进程**: OS 资源分配的基本单位, 拥有独立地址空间
- **线程**: CPU 调度的基本单位, 共享进程内存, 切换开销小

多线程目的: 充分利用多核 CPU + 提高程序吞吐量与响应速度

2 Java 线程 6 大生命周期（背诵版）

表格

复制

状态	触发条件	备注
NEW	<code>new Thread()</code> 未启动	——
RUNNABLE	<code>start()</code> 后	包含就绪 + 运行 两种 OS 状态
BLOCKED	抢 <code>synchronized</code> 失败	等锁
WAITING	<code>wait()/join()/LockSupport.park()</code>	无限期等通知
TIMED_WAITING	<code>sleep(long)/wait(long)/join(long)</code>	超时等通知
TERMINATED	<code>run()</code> 结束或异常退出	不可复活

jstack 实战：

bash

复制

```
jstack <pid> | grep -A 2 java.lang.Thread.State
```

3 创建线程的 4 种官方姿势（含虚拟线程）

表格

复制

方式	代码示例	特点	生产建议
继承 Thread	<code>class T extends Thread</code>	单继承 局限	✗ 测试/ 教学
实现 Runnable	<code>new Thread(() -> {...})</code>	无返回 值	✓ 常用

方式	代码示例	特点	生产建议
实现 Callable	<code>Future f = pool.submit(callable)</code>	有返回值	✅ 异步编排
线程池	<code>ThreadPoolExecutor</code>	复用 + 治理	✅ 唯一入口
虚拟线程	<code>Thread.ofVirtual().start(...)</code>	百万并发	✅ IO 密集

结论：生产永远用线程池或虚拟线程；前三种只出现在面试。

4 线程池 7 大参数（面试手写版）

java

复制

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(
    8, // corePoolSize 核心线程
    16, // maximumPoolSize 最大线程
    60L, TimeUnit.SECONDS, // keepAliveTime + unit 空闲存活
    new ArrayBlockingQueue<>(1000), // workQueue 有界队列
    new ThreadFactoryBuilder().setNameFormat("biz-%d").build(), // threadFactory
    new ThreadPoolExecutor.CallerRunsPolicy() // 拒绝策略
);
```

拒绝策略 4 选 1：

- Abort（抛异常）
- CallerRuns（调用者线程兜底）
- Discard（静默丢）
- DiscardOldest（丢旧任务）

陷阱： `Executors.newFixedThreadPool` 队列无界，OOM 风险。

5 并发工具包 (JUC) 速查

表格

复制

工具	作用	示例场景
CountDownLatch	等 N 线程完成	启动前加载多份配置
CyclicBarrier	循环汇合	多阶段并行计算
Semaphore	限流	接口 QPS 控制
Exchanger	两线程交换数据	单生产-单消费
CompletableFuture	异步编排	多服务并行调用
BlockingQueue	生产者-消费者	线程池底层队列

6 线程安全：synchronized 进化史

表格

复制

阶段	锁状态	特点
早期	重量级锁	内核态切换，慢
JDK6	偏向锁→轻量级锁→重量锁	无竞争偏向，CAS 自旋
JDK15	偏向锁默认关闭	撤销代价高，省 5 % CPU

结论：短临界区直接用 synchronized；长/复杂用 ReentrantLock（可中断、公平、条件队列）。

7 虚拟线程 (Project Loom, JDK21 正式)

- 创建：

java

复制

```
Thread vt = Thread.ofVirtual()
    .name("vt-")
    .start(() -> callRpc());
```

- 特点：JVM 级用户态线程，栈几百字节，百万级并发
- 适用：IO 密集（网关、RPC 聚合）
- 不适用：CPU 密集（计算、加密）

结构化并发：

java

复制

```
try (var scope = new StructuredTaskScope<Integer>()) {
    Future<Integer> f1 = scope.fork(() -> callServiceA());
    Future<Integer> f2 = scope.fork(() -> callServiceB());
    scope.join();           // 等全部完成
}
```

8 性能调优与观测（2025 云原生）

表格

复制

问题	工具	命令/参数
CPU 飙高	<code>top -H + jstack</code>	定位 TID → 十六进制 → 栈
死锁	<code>jcmd Thread.print</code>	自动死锁链
线程泄漏	<code>jstack wc -l</code>	看总量趋势
池打满	<code>ThreadPoolExecutor</code> MBean	<code>getActiveCount()</code>
虚拟线程	JFR 事件 <code>jdk.VirtualThreadStart</code>	JMC 可视化

9 线上案例 1：网关 5 w QPS 线程池打满

现象：RT 暴涨 → 拒绝异常

解决：

- `core=max=CPU×2`（避免伸缩）
- 队列换 `ArrayBlockingQueue(5000)`（有界）
- 拒绝策略 `CallerRuns`（兜底）

结果：RT 恢复 99.9 %。

10 线上案例 2：虚拟线程百万并发压测

结果：16 GB 内存，1 M 并发，P99 20 ms

关键：结构化并发 + IO 异步（Netty + Reactor）。

11 脑图速记（面试 2 分钟）

复制

Java 多线程

- ├ 状态：6 种，jstack 看
 - ├ 创建：Runnable > Callable > 线程池/虚拟线程
 - ├ 池：7 参数、有界队列、拒绝策略
 - ├ 安全：synchronized 锁升级、JUC 工具包
 - └ 虚拟线程：IO 密集、百万级、StructuredTaskScope
-

12 小结

掌握「6 大状态 → 4 种创建 → 7 参数线程池 → JUC 工具 → 虚拟线程 → 观测与调优」六件套，面试 2 分钟讲完、上线 5 分钟定位 **线程泄露/死锁/池打满** 问题。祝学习顺利，若对 **虚拟线程调度器**、**StructuredTaskScope**、**JUC StampedLock** 等想继续深挖，随时再来问！

一、线程基础与生命周期

1.1 什么是线程？

线程是进程中的一个执行单元，是 CPU 调度的基本单位。Java 中的线程是**用户线程**映射到操作系统线程（1:1 模型）。

- **进程 (Process)**：操作系统资源分配的基本单位；
- **线程 (Thread)**：进程内的执行流，共享进程内存空间；
- **协程 (Coroutine)**：用户态轻量级线程 (JDK 19+ Virtual Threads)。

1.2 线程状态详解 (Thread.State)

```
public enum State {  
    NEW,                // 新建状态: 创建了 Thread 对象, 但未调用 start()  
    RUNNABLE,           // 可运行状态: 线程在 JVM 中执行 (包括正在运行或等待资源)  
    BLOCKED,            // 阻塞状态: 等待监视器锁 (synchronized)  
    WAITING,            // 等待状态: 调用 Object.wait(), Thread.join(),  
    LockSupport.park()  
    TIMED_WAITING,      // 计时等待: sleep(), wait(timeout), join(timeout)  
    等  
    TERMINATED          // 终止状态: 线程执行完毕  
}
```

第一部分：线程的核心基础

1. 线程的创建与本质

两种创建方式的本质区别：

- **继承 Thread 类**：本质是多个独立的 Thread 子类实例，各自执行自己的 run 逻辑。资源不共享。
- **实现 Runnable 接口**：多个 Thread 实例可以共享同一个 Runnable 任务对象，天然地共享资源。

java

```
// 共享任务示例  
public class SharedTask implements Runnable {
```

```

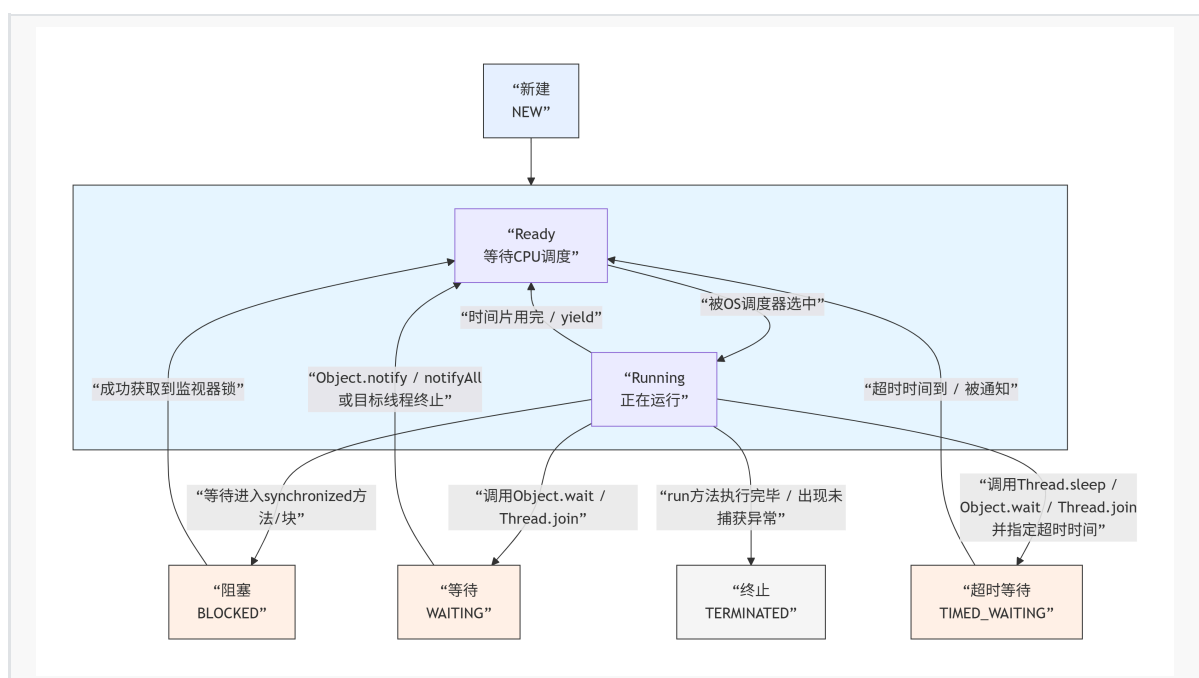
private int count = 0;
@Override
public void run() {
    for (int i = 0; i < 1000; i++) {
        count++; // 多个线程共享同一个SharedTask实例，共享count变量
    }
}
public int getCount() { return count; }
}

public static void main(String[] args) throws InterruptedException
{
    SharedTask sharedTask = new SharedTask(); // 一个任务对象
    Thread t1 = new Thread(sharedTask);
    Thread t2 = new Thread(sharedTask); // 两个线程共享同一个任务
    t1.start();
    t2.start();
    t1.join();
    t2.join();
    System.out.println(sharedTask.getCount()); // 结果可能小于2000，
    因为存在数据竞争
}

```

2. 线程的生命周期与状态转换

下图清晰地展示了Java线程的完整生命周期与状态转换的触发条件：



第二部分：线程同步的深度剖析

1. `synchronized` 的底层实现

`synchronized` 的锁是存储在 Java 对象头中的。在 JVM 中，对象在内存中的布局分为三部分：

1. 对象头
2. 实例数据
3. 对齐填充

其中，**对象头** 包含了锁状态、GC 年龄、哈希码等信息。`synchronized` 的锁升级过程就发生在对象头中。

锁升级过程 (JDK 6+ 的重要优化)：

1. **偏向锁**：假设只有一个线程访问同步块。当线程访问时，会在对象头和栈帧中记录偏向的线程 ID，以后该线程进入和退出同步块时不需要进行 CAS 操作来加锁和解锁。
2. **轻量级锁**：当有第二个线程尝试获取锁时，偏向锁会升级为轻量级锁。线程通过 **CAS 操作** 和 **自旋** 来尝试获取锁，避免直接进入重量级锁的阻塞。
3. **重量级锁**：当轻量级锁竞争激烈（线程自旋超过一定次数），会升级为重量级锁。此时未获取到锁的线程会**进入阻塞状态**，依赖于操作系统底层的互斥量来实现，开销最大。

2. `volatile` 的内存语义

`volatile` 的底层是通过 **内存屏障** 来实现的。

- **写操作**：JVM 会在 `volatile` 写之后插入一个 **StoreLoad** 屏障，强制将工作内存的修改刷新到主内存。
- **读操作**：JVM 会在 `volatile` 读之前插入一个 **LoadLoad** 屏障，强制从主内存重新加载数据。

这保证了：

1. **可见性**：一个线程修改了 `volatile` 变量，新值对其他线程立即可见。
 2. **禁止指令重排序**：屏障阻止了编译器和管理器的优化重排。
-

第三部分：JUC 包的基石——AQS

`java.util.concurrent.locks.AbstractQueuedSynchronizer` (AQS) 是构建 JUC 包中大多数同步器（如 `ReentrantLock`, `CountDownLatch`, `Semaphore`）的框架。

1. AQS 的核心思想

AQS 使用了一个 **volatile int state** 成员变量来表示同步状态，并通过一个内置的 **FIFO 队列** 来管理获取锁失败的线程。

java

```
// 简化的 AQS 工作原理
public abstract class AbstractQueuedSynchronizer {
    private volatile int state; // 核心状态变量
    private transient volatile Node head; // 队列头
    private transient volatile Node tail; // 队列尾

    // 获取锁的模板方法
    public final void acquire(int arg) {
        if (!tryAcquire(arg) && // 尝试获取锁（由子类实现）
            acquireQueued(addWaiter(Node.EXCLUSIVE), arg)) // 获取
            失败，加入队列
            selfInterrupt();
    }

    protected boolean tryAcquire(int arg) { // 需要子类重写
        throw new UnsupportedOperationException();
    }
}
```

2. `ReentrantLock` 与 AQS

`ReentrantLock` 内部有一个继承自 AQS 的同步器。

- `state == 0`：锁未被持有。
- `state == 1`：锁被一个线程持有。
- `state > 1`：锁被同一个线程重入。

`tryAcquire` 的逻辑：

1. 如果 `state == 0`，尝试用 CAS 将 state 设为 1，成功则获取锁。
2. 如果当前线程是已持有锁的线程，则 `state++`（重入）。

3. 否则，获取失败。

第四部分：原子类与 CAS

1. 什么是 CAS?

CAS 是 **Compare-And-Swap** 的缩写，是一种**无锁**的原子操作。它包含三个操作数：

- 内存位置
- 期望的原值
- 新值

操作逻辑：当且仅当内存位置的值与期望的原值相等时，处理器才会将该位置的值更新为新值，否则不做任何操作。整个操作是一个原子指令。

2. 原子类的实现

以 `AtomicInteger` 为例：

java

```
public class AtomicInteger extends Number {
    private volatile int value; // 使用 volatile 保证可见性
    private static final long valueOffset; // value 字段的内存偏移地址

    static {
        try {
            valueOffset = unsafe.objectFieldOffset
                (AtomicInteger.class.getDeclaredField("value"));
        } catch (Exception ex) { throw new Error(ex); }
    }

    public final int incrementAndGet() {
        return unsafe.getAndAddInt(this, valueOffset, 1) + 1;
    }
}
```

在 `Unsafe.getAndAddInt` 方法内部，是一个 **CAS 自旋循环**：

java

```
public final int getAndAddInt(Object o, long offset, int delta) {
    int v;
    do {
        v = getIntVolatile(o, offset); // 读取当前值
    } while (!compareAndSwapInt(o, offset, v, v + delta)); // CAS
    失败则循环重试
    return v;
}
```

3. CAS 的 ABA 问题与解决

- **问题：**一个变量初始值为 A，线程1准备将其改为 C。在此期间，线程2将其先改为 B，又改回 A。线程1的 CAS 操作会成功，但它不知道值已经历过 A->B->A 的变化。
- **解决方案：**`AtomicStampedReference`，它通过一个版本戳来记录变量的变化。

第五部分：线程池的深度解析

1. 核心参数详解

`ThreadPoolExecutor` 的构造方法：

java

```
public ThreadPoolExecutor(
    int corePoolSize,        // 核心线程数
    int maximumPoolSize,     // 最大线程数
    long keepAliveTime,      // 非核心线程空闲存活时间
    TimeUnit unit,           // 时间单位
    BlockingQueue<Runnable> workQueue, // 工作队列
    ThreadFactory threadFactory, // 线程工厂
    RejectedExecutionHandler handler // 拒绝策略
)
```

2. 任务执行流程

1. 提交任务。
2. 如果当前运行线程数 < `corePoolSize`，创建新线程执行任务。
3. 否则，将任务放入 `workQueue`。

4. 如果队列已满，且运行线程数 < `maximumPoolSize`，创建新线程执行任务。
5. 否则，触发 **拒绝策略**。

3. 四种拒绝策略

- `AbortPolicy`（默认）：抛出 `RejectedExecutionException`。
- `CallerRunsPolicy`：由调用者线程（提交任务的线程）自己执行该任务。
- `DiscardPolicy`：直接丢弃任务，不做任何处理。
- `DiscardOldestPolicy`：丢弃队列中最老的任务，然后重新尝试提交当前任务。

4. 工作队列类型

- `SynchronousQueue`：不存储元素的队列。每个插入操作必须等待另一个线程的移除操作。用于 `CachedThreadPool`。
- `LinkedBlockingQueue`：无界队列（除非构造时指定容量）。用于 `FixedThreadPool` 和 `SingleThreadExecutor`。
- `ArrayBlockingQueue`：有界队列，需要指定容量。

第六部分：高级并发模式与最佳实践

1. 生产者-消费者模式

使用 `BlockingQueue` 可以轻松实现。

java

```
public class ProducerConsumerExample {
    private final BlockingQueue<Integer> queue = new
    LinkedBlockingQueue<>(10);

    class Producer implements Runnable {
        public void run() {
            try {
                int value = 0;
                while (true) {
                    queue.put(value); // 队列满时会自动阻塞
                    System.out.println("生产: " + value);
                    value++;
                    Thread.sleep(100);
                }
            } catch (InterruptedException e) {
```

```

        Thread.currentThread().interrupt();
    }
}

class Consumer implements Runnable {
    public void run() {
        try {
            while (true) {
                Integer value = queue.take(); // 队列空时会自动阻塞
                System.out.println("消费: " + value);
                Thread.sleep(150);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

```

2. ThreadLocal 原理与内存泄漏

`ThreadLocal` 为每个线程提供了一个独立的变量副本，避免了共享问题。

原理：每个 `Thread` 对象内部都有一个 `ThreadLocalMap`，它以 `ThreadLocal` 对象为 Key，存储线程的变量副本。

内存泄漏风险：

- `ThreadLocalMap` 的 Key 是弱引用指向 `ThreadLocal` 对象。
- 如果 `ThreadLocal` 外部强引用被置为 `null`，GC 时 Key 会被回收，但 Value 仍然是强引用，只要线程不终止（如线程池中的线程），Value 就无法被回收。
- **解决方案：**使用完 `ThreadLocal` 后，必须调用 `remove()` 方法清理 Entry。

java

```
public class ThreadLocalExample {
    private static final ThreadLocal<SimpleDateFormat> formatter =
        ThreadLocal.withInitial(() -> new
SimpleDateFormat("yyyy-MM-dd"));

    public static void main(String[] args) {
        // 使用 formatter.get()
        // ...
        // 使用完毕后清理，防止内存泄漏
        formatter.remove();
    }
}
```

总结

Java 多线程的深度知识体系：

1. **基础机制**：线程创建、生命周期、`synchronized`、`volatile`。
2. **JMM (Java内存模型)**：Happens-Before、内存屏障、可见性、有序性。
3. **无锁编程**：CAS、原子类、ABA 问题。
4. **同步框架**：AQS 原理、`ReentrantLock`、`ReentrantReadWriteLock`。
5. **并发工具**：`CountDownLatch`、`CyclicBarrier`、`Semaphore`、`Exchanger`。
6. **线程池**：核心参数、执行流程、拒绝策略、队列类型。
7. **并发容器**：`ConcurrentHashMap`、`CopyOnWriteArrayList`、`ConcurrentLinkedQueue`。
8. **高级主题**：`ThreadLocal`、`Fork/Join` 框架、`CompletableFuture`。

掌握这些内容，意味着你不仅能够应对多线程开发中的各种场景，更能理解其底层运作机制，从而写出高效、健壮、可维护的并发程序。这是一个从“会用”到“精通”的飞跃。

Java 多线程详解：从基础原理到高并发实战

Java 多线程是构建高性能、高并发应用程序的核心技术。它不仅能充分利用多核 CPU，还能提升用户体验和系统吞吐量。本文将从**线程生命周期**、**并发原理**、**核心 API**、**实战应用**四个维度，全面深入解析 Java 多线程编程。

一、线程基础与生命周期

1.1 什么是线程？

线程是进程中的一个执行单元，是 CPU 调度的基本单位。Java 中的线程是**用户线程**映射到**操作系统线程**（1:1 模型）。

- **进程 (Process)**：操作系统资源分配的基本单位；
- **线程 (Thread)**：进程内的执行流，共享进程内存空间；
- **协程 (Coroutine)**：用户态轻量级线程（JDK 19+ Virtual Threads）。

1.2 线程状态详解 (Thread.State)

```
public enum State {  
    NEW,                // 新建状态: 创建了 Thread 对象, 但未调用 start()  
    RUNNABLE,           // 可运行状态: 线程在 JVM 中执行 (包括正在运行或等待资源)  
    BLOCKED,            // 阻塞状态: 等待监视器锁 (synchronized)  
    WAITING,            // 等待状态: 调用 Object.wait(), Thread.join(),  
                        // LockSupport.park()  
    TIMED_WAITING,      // 计时等待: sleep(), wait(timeout), join(timeout)  
                        // 等  
    TERMINATED          // 终止状态: 线程执行完毕  
}
```

1.3 线程创建的四种方式

方式一：继承 Thread 类

```
public class MyThread extends Thread {  
    @Override  
    public void run() {  
        System.out.println("Thread running: " +  
            Thread.currentThread().getName());  
    }  
}
```



```
public static void main(String[] args) {  
    MyThread t = new MyThread();  
    t.start(); // 启动线程，不能直接调用 run()  
}  
}
```

⚠️ **缺点：**Java 单继承，扩展性差。

方式二：实现 Runnable 接口（推荐）

```
public class MyRunnable implements Runnable {  
    @Override  
    public void run() {  
        System.out.println("Runnable running: " +  
Thread.currentThread().getName());  
    }  
  
    public static void main(String[] args) {  
        Thread t = new Thread(new MyRunnable());  
        t.start();  
  
        // Lambda 表达式 (JDK 8+)  
        new Thread(() -> System.out.println("Lambda  
thread")).start();  
    }  
}
```

方式三：实现 Callable 接口（有返回值）

```
import java.util.concurrent.*;  
  
public class MyCallable implements Callable<String> {  
    @Override  
    public String call() throws Exception {  
        Thread.sleep(1000);  
        return "Task completed!";  
    }  
  
    public static void main(String[] args) throws  
ExecutionException, InterruptedException {  
        ExecutorService executor =  
Executors.newSingleThreadExecutor();  
        Future<String> future = executor.submit(new MyCallable());  
    }  
}
```

```
        System.out.println("Task result: " + future.get()); // 阻塞等待结果
        executor.shutdown();
    }
}
```

方式四：使用线程池（最佳实践）

```
public class ThreadPoolDemo {
    public static void main(String[] args) throws
        InterruptedException {
        ExecutorService executor =
            Executors.newFixedThreadPool(2);

        // 提交任务
        Future<String> future = executor.submit(() -> {
            Thread.sleep(1000);
            return "Result";
        });

        System.out.println("Future result: " + future.get());
        executor.shutdown();
    }
}
```

二、并发三大核心问题

2.1 可见性 (Visibility)

问题：一个线程修改变量后，其他线程可能看不到最新值。

```

public class visibilityProblem {
    private static boolean flag = false;

    public static void main(String[] args) throws
InterruptedException {
        new Thread(() -> {
            while (!flag) { } // 可能永远不退出 (CPU 缓存)
            System.out.println("Flag is true!");
        }).start();

        Thread.sleep(1000);
        flag = true; // 主线程修改
    }
}

```

解决方案：

1. **volatile** 关键字：

```
private static volatile boolean flag = false; // 保证可见性
```

2. **synchronized**

```
synchronized (lock) { flag = true; } // 解锁时刷新到主存
```

3. **Atomic** 类：

```
private static AtomicBoolean atomicFlag = new
AtomicBoolean(false);
```

2.2 原子性 (Atomicity)

问题：复合操作 (如 `i++`) 不是原子的，可能导致结果不正确。

```

public class AtomicityProblem {
    private static int count = 0;

    public static void main(String[] args) throws
InterruptedException {
        Thread[] threads = new Thread[10];
        for (int i = 0; i < 10; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++) {
                    count++; // 非原子操作：读-改-写
                }
            });
        }
    }
}

```

```

        }
    });
}

for (Thread t : threads) t.start();
for (Thread t : threads) t.join();

System.out.println("Expected: 10000, Actual: " + count);
// 结果 < 10000
}
}

```

解决方案:

1. **synchronized:**

```
private static synchronized void increment() { count++; }
```

2. **AtomicInteger:**

```
private static AtomicInteger atomicCount = new AtomicInteger(0);
atomicCount.incrementAndGet(); // CAS 原子操作
```

3. **ReentrantLock:**

```
private static final Lock lock = new ReentrantLock();
lock.lock(); try { count++; } finally { lock.unlock(); }
```

2.3 有序性 (Ordering)

问题: 编译器/CPU 重排序可能导致程序执行顺序与代码顺序不一致。

```
public class OrderingProblem {
    private static int x = 0, y = 0;
    private static int a = 0, b = 0;

    public static void main(String[] args) throws
    InterruptedException {
        int count = 0;
        for (int i = 0; i < 100000; i++) {
            x = y = a = b = 0; // 重置

            Thread t1 = new Thread(() -> {
                a = 1; x = b; // 可能重排序为: x=b; a=1;
            });

```

```

        Thread t2 = new Thread() -> {
            b = 1; y = a; // 可能重排序为: y=a; b=1;
        });

        t1.start(); t2.start();
        t1.join(); t2.join();

        if (x == 0 && y == 0) count++; // 可能发生!
    }
    System.out.println("x=0,y=0 occurred " + count + "
times");
}
}

```

解决方案:

- **volatile**: 写后读的顺序性;
- **synchronized**: 解锁前的所有操作 hb 加锁后的操作。

三、synchronized 关键字深度解析

3.1 synchronized 的三种用法

```

public class SynchronizedDemo {
    private final Object lock = new Object();

    // 1. 修饰实例方法（锁当前实例）
    public synchronized void method1() {
        System.out.println("Instance method locked: " + this);
    }

    // 2. 修饰静态方法（锁当前类）
    public static synchronized void method2() {
        System.out.println("Static method locked: " +
SynchronizedDemo.class);
    }

    // 3. 修饰代码块（锁指定对象）
    public void method3() {
        synchronized (lock) {
            System.out.println("Block locked: " + lock);
        }
    }
}

```

3.2 synchronized 实现原理

JVM 层面

- `monitorenter` / `monitorexit` 指令;
- 对象头 (Object Header) 包含 Mark Word (锁信息)。

锁升级机制 (JDK 1.6+)

无锁 → 偏向锁 → 轻量级锁 → 重量级锁

↑ ↑ ↑ ↑

无竞争 单线程竞争 多线程竞争 操作系统互斥

- **偏向锁**: 偏向第一个获取锁的线程, 减少 CAS 开销;
- **轻量级锁**: 自旋获取锁, 避免线程阻塞;
- **重量级锁**: 线程阻塞, OS 调度。

3.3 死锁问题与避免

```
public class DeadlockDemo {
    private static final Object lock1 = new Object();
    private static final Object lock2 = new Object();

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            synchronized (lock1) {
                System.out.println("Thread 1: lock1 acquired");
                try { Thread.sleep(100); } catch
                (InterruptedException e) { }
                synchronized (lock2) { // 等待 lock2
                    System.out.println("Thread 1: lock2
acquired");
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (lock2) {
                System.out.println("Thread 2: lock2 acquired");
                try { Thread.sleep(100); } catch
                (InterruptedException e) { }
                synchronized (lock1) { // 等待 lock1
```

```

        System.out.println("Thread 2: lock1
acquired");
    }
}
});

t1.start(); t2.start();
}
}

```

避免死锁策略：

1. **固定锁顺序**：所有线程按相同顺序获取锁；
2. **超时机制**：`tryLock(timeout)`；
3. **死锁检测**：`ThreadMXBean.findDeadlockedThreads()`。

四、JUC 并发包详解

4.1 ReentrantLock（可重入锁）

```

import java.util.concurrent.locks.ReentrantLock;

public class ReentrantLockDemo {
    private final ReentrantLock lock = new ReentrantLock();
    private int count = 0;

    public void increment() {
        lock.lock();
        try {
            count++;
        } finally {
            lock.unlock(); // 必须在 finally 中释放
        }
    }

    // 尝试获取锁（非阻塞）
    public boolean tryIncrement() {
        if (lock.tryLock()) {
            try {
                count++;
                return true;
            } finally {
                lock.unlock();
            }
        }
    }
}

```

```

    }
    return false;
}

// 可中断的锁获取
public void interruptibleIncrement() throws
InterruptedException {
    lock.lockInterruptibly();
    try {
        count++;
    } finally {
        lock.unlock();
    }
}
}

```

ReentrantLock vs synchronized:

特性	synchronized	ReentrantLock
可中断	否	是
超时获取	否	是
公平锁	否	是（可选）
条件变量	wait/notify	Condition
性能	JDK 1.6+ 优化后接近	高并发下更优

4.2 ReadWriteLock（读写锁）

```

import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class ReadWriteLockDemo {
    private final ReadWriteLock rwLock = new
ReentrantReadWriteLock();
    private final Map<String, String> cache = new HashMap<>();

    public String get(String key) {
        rwLock.readLock().lock();
        try {
            return cache.get(key);
        } finally {
            rwLock.readLock().unlock();
        }
    }
}

```



```

    }
}

public void put(String key, String value) {
    rwLock.writeLock().lock();
    try {
        cache.put(key, value);
    } finally {
        rwLock.writeLock().unlock();
    }
}
}

```

4.3 Condition（条件变量）

```

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class ConditionDemo {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notEmpty = lock.newCondition();
    private final Condition notFull = lock.newCondition();
    private final Queue<String> queue = new ArrayDeque<>();
    private final int capacity = 10;

    public void put(String item) throws InterruptedException {
        lock.lock();
        try {
            while (queue.size() == capacity) {
                notFull.await(); // 队列满时等待
            }
            queue.offer(item);
            notEmpty.signal(); // 通知消费者
        } finally {
            lock.unlock();
        }
    }

    public String take() throws InterruptedException {
        lock.lock();
        try {
            while (queue.isEmpty()) {
                notEmpty.await(); // 队列空时等待
            }
        }
    }
}

```

```

        String item = queue.poll();
        notFull.signal(); // 通知生产者
        return item;
    } finally {
        lock.unlock();
    }
}
}

```

五、JUC 同步工具类

5.1 CountdownLatch (倒计时门闩)

```

import java.util.concurrent.CountDownLatch;

public class CountdownLatchDemo {
    public static void main(String[] args) throws
        InterruptedException {
        int threadCount = 3;
        CountDownLatch latch = new CountDownLatch(threadCount);

        for (int i = 0; i < threadCount; i++) {
            final int taskId = i;
            new Thread(() -> {
                System.out.println("Task " + taskId + "
starting...");
                try { Thread.sleep(1000); } catch
                (InterruptedException e) { }
                System.out.println("Task " + taskId + "
completed");
                latch.countDown(); // 计数器减1
            }).start();
        }

        System.out.println("waiting for all tasks to
complete...");
        latch.await(); // 等待计数器归零
        System.out.println("All tasks completed!");
    }
}

```

5.2 CyclicBarrier (循环屏障)

```
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;

public class CyclicBarrierDemo {
    public static void main(String[] args) {
        int threadCount = 3;
        CyclicBarrier barrier = new CyclicBarrier(threadCount, ()
-> {
            System.out.println("All threads reached the
barrier!");
        });

        for (int i = 0; i < threadCount; i++) {
            final int workerId = i;
            new Thread(() -> {
                try {
                    System.out.println("worker " + workerId + "
doing work...");
                    Thread.sleep(1000);
                    System.out.println("worker " + workerId + "
waiting at barrier");
                    barrier.await(); // 等待其他线程
                    System.out.println("worker " + workerId + "
passed barrier");
                } catch (InterruptedException |
BrokenBarrierException e) {
                    e.printStackTrace();
                }
            }).start();
        }
    }
}
```

5.3 Semaphore (信号量)

```
import java.util.concurrent.Semaphore;

public class SemaphoreDemo {
    private static final Semaphore semaphore = new Semaphore(2);
    // 最多2个线程同时访问
```

```

        public static void main(String[] args) throws
InterruptedException {
            for (int i = 0; i < 5; i++) {
                final int clientId = i;
                new Thread(() -> {
                    try {
                        semaphore.acquire(); // 获取许可
                        System.out.println("Client " + clientId + "
accessing resource");
                        Thread.sleep(2000); // 模拟资源使用
                        System.out.println("Client " + clientId + "
releasing resource");
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    } finally {
                        semaphore.release(); // 释放许可
                    }
                }).start();
            }
        }
    }
}

```

六、并发集合类详解

6.1 ConcurrentHashMap (高性能并发 Map)

```

import java.util.concurrent.ConcurrentHashMap;

public class ConcurrentHashMapDemo {
    private static final ConcurrentHashMap<String, String> map =
new ConcurrentHashMap<>();

    public static void main(String[] args) throws
InterruptedException {
        // 线程安全的 put/get
        Thread writer = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                map.put("key" + i, "value" + i);
            }
        });

        Thread reader = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                String value = map.get("key" + i);
            }
        });
    }
}

```

```

        if (value != null) {
            System.out.println("Read: " + value);
        }
    }
});

writer.start(); reader.start();
writer.join(); reader.join();
}
}

```

ConcurrentHashMap vs Hashtable vs Collections.synchronizedMap:

特性	ConcurrentHashMap	Hashtable	Collections.synchronizedMap
锁粒度	分段锁 (JDK 7) / CAS + synchronized (JDK 8+)	全表锁	全表锁
性能	高	低	低
null 值	不允许	不允许	允许

6.2 BlockingQueue (阻塞队列)

```

import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;

public class ProducerConsumerDemo {
    private static final BlockingQueue<String> queue = new
ArrayBlockingQueue<>(10);

    public static void main(String[] args) {
        // 生产者
        Thread producer = new Thread(() -> {
            for (int i = 0; i < 20; i++) {
                try {
                    String item = "Item-" + i;
                    queue.put(item); // 队列满时阻塞
                    System.out.println("Produced: " + item);
                } catch (InterruptedException e) {

```

```

        Thread.currentThread().interrupt();
    }
}

// 消费者
Thread consumer = new Thread(() -> {
    for (int i = 0; i < 20; i++) {
        try {
            String item = queue.take(); // 队列空时阻塞
            System.out.println("Consumed: " + item);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
});

producer.start(); consumer.start();
}
}

```

BlockingQueue 实现对比:

实现	特点	适用场景
ArrayBlockingQueue	有界、FIFO、公平锁可选	生产者-消费者
LinkedBlockingQueue	可选有界、FIFO	高并发
SynchronousQueue	不存储元素、直接传递	高性能传递
PriorityBlockingQueue	优先级队列	任务调度
DelayQueue	延迟元素	定时任务

七、原子类详解

7.1 基本原子类

```

import java.util.concurrent.atomic.AtomicInteger;

public class AtomicDemo {
    private static final AtomicInteger atomicInt = new
AtomicInteger(0);

```

```

    public static void main(String[] args) throws
InterruptedException {
        Thread[] threads = new Thread[10];
        for (int i = 0; i < 10; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++) {
                    atomicInt.incrementAndGet(); // 原子操作
                }
            });
        }

        for (Thread t : threads) t.start();
        for (Thread t : threads) t.join();

        System.out.println("Atomic count: " + atomicInt.get()); //
10000
    }
}

```

原子类体系：

- **基本类型：** `AtomicInteger`, `AtomicLong`, `AtomicBoolean`
- **数组类型：** `AtomicIntegerArray`, `AtomicLongArray`
- **引用类型：** `AtomicReference`, `AtomicMarkableReference`
- **更新器：** `AtomicIntegerFieldUpdater`, `AtomicReferenceFieldUpdater`
- **累加器：** `LongAdder`, `DoubleAdder` (JDK 8+, 高并发下性能优于 Atomic)

7.2 LongAdder vs AtomicInteger

```

// 高并发场景下，LongAdder 性能更优
private static final LongAdder longAdder = new LongAdder();
private static final AtomicInteger atomicInt = new
AtomicInteger(0);

// LongAdder 通过分段累加减少竞争
longAdder.increment(); // 内部有多个 cell
// atomicInt.incrementAndGet(); // CAS 竞争激烈

```

八、线程池详解

8.1 线程池核心参数

```
public class ThreadPoolDemo {
    public static void main(String[] args) {
        // 自定义线程池（推荐）
        ThreadPoolExecutor executor = new ThreadPoolExecutor(
            2,                    // 核心线程数
            4,                    // 最大线程数
            60L,                  // 空闲线程存活时间
            TimeUnit.SECONDS,     // 时间单位
            new LinkedBlockingQueue<>(100), // 工作队列
            Thread::new,          // 线程工厂
            new ThreadPoolExecutor.CallerRunsPolicy() // 拒绝策略
        );

        // 提交任务
        for (int i = 0; i < 10; i++) {
            final int taskId = i;
            executor.submit(() -> {
                System.out.println("Task " + taskId + " executed
by " +
                                Thread.currentThread().getName());
                try { Thread.sleep(1000); } catch
(InterruptedExecution e) { }
            });
        }

        executor.shutdown(); // 不接受新任务，等待现有任务完成
    }
}
```

8.2 拒绝策略

策略	行为	适用场景
AbortPolicy	抛出 RejectedExecutionException	默认，快速失败
CallerRunsPolicy	调用者线程执行任务	保护系统不被压垮
DiscardPolicy	直接丢弃任务	允许丢失

策略	行为	适用场景
DiscardOldestPolicy	丢弃队列头部任务，重新提交	丢弃旧任务

8.3 Executors 工厂方法（谨慎使用）

```
// 1. 固定大小线程池（推荐）
ExecutorService fixedPool = Executors.newFixedThreadPool(4);

// 2. 缓存线程池（不推荐，可能创建过多线程）
ExecutorService cachedPool = Executors.newCachedThreadPool();

// 3. 单线程池
ExecutorService singlePool = Executors.newSingleThreadExecutor();

// 4. 定时任务线程池
ScheduledExecutorService scheduledPool =
    Executors.newScheduledThreadPool(2);
scheduledPool.scheduleAtFixedRate(() -> {
    System.out.println("Scheduled task");
}, 0, 1, TimeUnit.SECONDS);
```

⚠ Executors 的陷阱：

- `newFixedThreadPool` 使用无界队列，可能导致 OOM；
- `newCachedThreadPool` 最大线程数为 `Integer.MAX_VALUE`，可能创建过多线程。

九、CompletableFuture（异步编程）

9.1 基本用法

```
import java.util.concurrent.CompletableFuture;

public class CompletableFutureDemo {
    public static void main(String[] args) throws Exception {
        // 异步执行任务
        CompletableFuture<String> future = CompletableFuture
            .supplyAsync(() -> {
                try { Thread.sleep(1000); } catch
                (InterruptedException e) { }
                return "Hello Async";
            })
    }
```

```

        .thenApply(result -> result + " world")
        .thenApply(String::toUpperCase);

    System.out.println("Result: " + future.get()); // 阻塞等待

    // 组合多个异步任务
    CompletableFuture<String> future1 =
    CompletableFuture.supplyAsync(() -> "Task1");
    CompletableFuture<String> future2 =
    CompletableFuture.supplyAsync(() -> "Task2");

    CompletableFuture<String> combined =
    future1.thenCombine(future2, (s1, s2) -> s1 + " + " + s2);
    System.out.println("Combined: " + combined.get());
}
}

```

9.2 异常处理

```

CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> {
        throw new RuntimeException("Error occurred");
    })
    .exceptionally(throwable -> {
        System.out.println("Caught exception: " +
throwable.getMessage());
        return "Default value";
    });

```

十、虚拟线程 (Virtual Threads)

JDK 19+ 引入的**轻量级线程**，极大简化了并发编程。

```

// JDK 19+ (Preview)
public class VirtualThreadDemo {
    public static void main(String[] args) throws
InterruptedException {
        // 创建虚拟线程
        Thread virtualThread = Thread.startVirtualThread(() -> {
            System.out.println("Running on virtual thread: " +
Thread.currentThread());

```

```

        try { Thread.sleep(1000); } catch
        (InterruptedException e) { }
    });

    virtualThread.join();

    // 或使用线程池
    try (ExecutorService executor =
    Executors.newVirtualThreadPerTaskExecutor()) {
        for (int i = 0; i < 100; i++) {
            final int taskId = i;
            executor.submit(() -> {
                System.out.println("Task " + taskId + " on
virtual thread");
                try { Thread.sleep(1000); } catch
                (InterruptedException e) { }
            });
        }
    } // 自动关闭
}
}

```

虚拟线程 vs 平台线程：

特性	虚拟线程	平台线程
创建开销	极低（毫秒级）	高（操作系统调度）
数量限制	百万级	千级
阻塞行为	自动挂起，不占用平台线程	阻塞平台线程
适用场景	IO 密集型	CPU 密集型

十一、并发设计模式

11.1 单例模式（线程安全）

```

public class Singleton {
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) { // 第一次检查

```

```

        synchronized (Singleton.class) {
            if (instance == null) { // 第二次检查
                instance = new Singleton(); // volatile 防止重
排序
            }
        }
    }
    return instance;
}
}

// 或使用枚举（推荐）
public enum SingletonEnum {
    INSTANCE;
    public void doSomething() { }
}

```

11.2 生产者-消费者模式

```

public class ProducerConsumerPattern {
    private final Queue<String> queue = new ArrayBlockingQueue<>
(10);

    class Producer implements Runnable {
        @Override
        public void run() {
            try {
                for (int i = 0; i < 100; i++) {
                    queue.put("Item-" + i);
                    System.out.println("Produced: Item-" + i);
                    Thread.sleep(100);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }

    class Consumer implements Runnable {
        @Override
        public void run() {
            try {
                while (true) {
                    String item = queue.take();

```

```

        System.out.println("Consumed: " + item);
        Thread.sleep(200);
    }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
}
}

```

11.3 Future 模式

```

public class FuturePattern {
    public static void main(String[] args) throws
        InterruptedException, ExecutionException {
        FutureTask<String> futureTask = new FutureTask<>(() -> {
            Thread.sleep(2000);
            return "Result after 2 seconds";
        });

        new Thread(futureTask).start();

        System.out.println("Doing other work...");
        System.out.println("Future result: " + futureTask.get());
    }
}

```

十二、性能调优与最佳实践

12.1 性能监控

```

// 使用 JMH 进行微基准测试
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
public class PerformanceTest {
    @Benchmark
    public void testSynchronized() {
        synchronized (this) {
            // 测试代码
        }
    }

    @Benchmark

```

```
public void testReentrantLock() {
    lock.lock();
    try {
        // 测试代码
    } finally {
        lock.unlock();
    }
}
```

12.2 最佳实践清单

场景	推荐方案
高并发读多写少	ConcurrentHashMap + ReadWriteLock
计数器	LongAdder > AtomicLong > synchronized
线程池	自定义配置，避免 Executors 默认池
异步处理	CompletableFuture + 自定义线程池
锁粒度	分段锁、读写分离
IO 密集型	虚拟线程 (JDK 19+)
CPU 密集型	平台线程，线程数 = CPU 核数

12.3 常见陷阱与避免

陷阱	避免方式
线程安全的误解	容器线程安全 ≠ 业务逻辑线程安全
死锁	固定锁顺序、使用超时机制
活锁	随机退避、优先级机制
饥饿	公平锁、合理分配资源
上下文切换	减少锁竞争、使用无锁数据结构

十三、总结：多线程核心要点

维度	关键点
三大问题	可见性（volatile）、原子性（synchronized/Atomic）、有序性（happens-before）
同步机制	synchronized（简单）、ReentrantLock（灵活）、JUC 工具类
并发集合	ConcurrentHashMap、BlockingQueue、CopyOnWriteArrayList
线程池	合理配置核心/最大线程数、队列大小、拒绝策略
异步编程	CompletableFuture、响应式编程（Reactor）
未来趋势	虚拟线程、结构化并发（Structured Concurrency）

 **一句话理解多线程：**
“并发是双刃剑，正确使用提升性能，错误使用引发灾难。核心是理解 JMM、掌握同步机制、合理使用并发工具。”

如需进一步了解：

- 如何实现一个自定义的阻塞队列？
- CompletableFuture 的线程池配置？
- JVM 并发调优实战？
- 虚拟线程在 Web 应用中的应用？

